



**Module Name:** *Data Mining*

|                              |                                                                                        |
|------------------------------|----------------------------------------------------------------------------------------|
| <b>Assignment No./Title</b>  | <b>Assignment (Individual)</b><br><b>30% (ASSIGNMENT 2 – EXECUTIVE SUMMARY – MLO2)</b> |
| <b>Course Tutor/Lecturer</b> | Ts. Dr Anbuselvan Sangodiah/Jafhate Edward                                             |
| <b>Section No</b>            | <i>Section 04</i>                                                                      |

# Khor

*Certify that this assignment is entirely my own work, except where I have given fully documented references to the work of others, and that the material contained in this assignment has not previously been submitted for assessment in any other formal course of study. I have also read and accept the content of the **documentation and submission instructions section** of the assignment 2 question.*

|                     |                      |
|---------------------|----------------------|
| <b>Marks/Grade:</b> | <b>Evaluated by:</b> |
|---------------------|----------------------|

\* Please include this cover page for your project submission

## Table of Contents

|                                              |    |
|----------------------------------------------|----|
| 1. Implementation of Apriori Algorithm ..... | 2  |
| 1.1 Preparation of Apriori Algorithm.....    | 2  |
| 1.2 Single-Level Association Rule .....      | 5  |
| 1.3 Negative Association Rule.....           | 10 |
| 1.4 Multi-Level Association Rule.....        | 15 |
| 2. Implementation of FP Algorithm .....      | 18 |
| 2.1 Preparation of FP-Growth.....            | 18 |
| 2.2 Single-dimensional Association Rule..... | 20 |
| 2.3 Multi-Level Association Rule.....        | 23 |
| 3. Summary of Observations.....              | 28 |
| 3.1 Key Finding .....                        | 28 |
| 3.2 Recommendation and Conclusion .....      | 31 |
| References .....                             | 32 |
| Appendix.....                                | 34 |
| Appendix A.....                              | 34 |
| Appendix B.....                              | 36 |
| Appendix C .....                             | 37 |
| Appendix D .....                             | 38 |
| .....                                        | 38 |

## 1. Implementation of Apriori Algorithm

Apriori Algorithm is an unsupervised method that can assist in generating an association rule from a large dataset. The significance of association rule entails that if a certain action X occurs, then action Y also happens with a certain probability. The rules can be represented by  $X \rightarrow Y$  (Elavarasan, 2018).

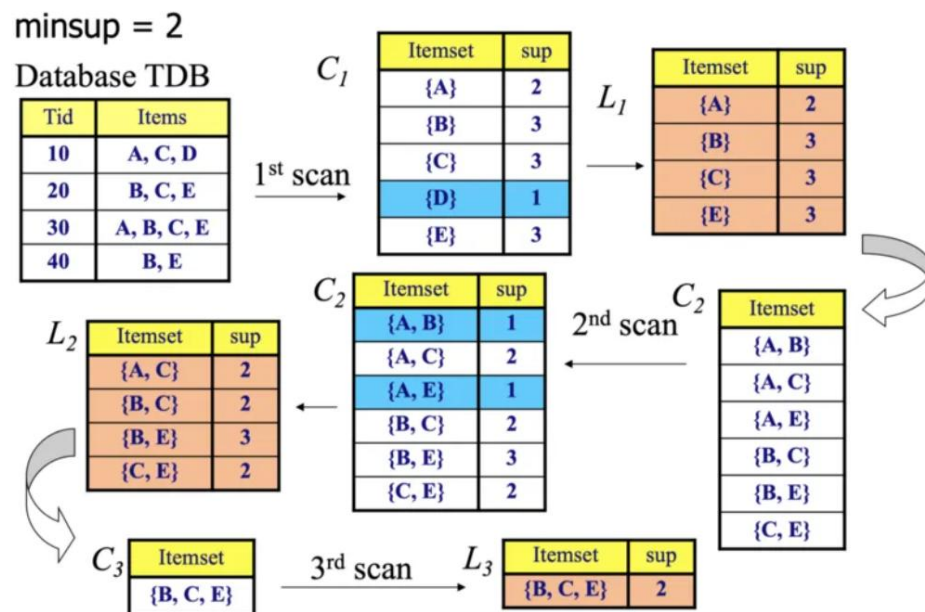


Figure 1: Apriori Algorithm

### 1.1 Preparation of Apriori Algorithm

In order to ensure the efficiency and accuracy of the association rule result, several preparations must be undertaken prior to the implementation of the Apriori Algorithm. These steps entail importing the required library in Python, setting up the table view in PyCharm for readability and loading the dataset into python.

#### Step 1: Importation of Necessary Libraries

```
import networkx as nx
import pandas as pd
import numpy as np
from apyori import apriori as ap
import matplotlib.pyplot as plt
```

Figure 2: Import libraries for Apriori Algorithm

- networkx
  - A Python packages that is used to generate a graph
- pandas
  - A powerful Python library used to manipulate and analyses the data
- numpy
  - A scientific-computing Python package that can create, access and operate the array in Numpy.
- apriori
  - A Python library that is essential to implement the Apriori Algorithm
- matplotlib.pyplot
  - This library serves as a state-based interface, and it offers a plotting method that is implicit and resembles MATLAB

## Step 2: Set up the table view

```
desired_width = 5000
pd.set_option('display.width', desired_width)
pd.set_option('display.max_columns', None)
pd.set_option('display.max_colwidth', None)
pd.set_option('display.max_rows', None)
np.set_printoptions(linewidth=desired_width)
```

Figure 3:PyCharm view setup

The Figure above shows the configuration of the data frame's display setting in PyCharm to improve the readability by ensuring the full view of the table.

## Step 3: Loading the Dataset

```
AR_data = pd.read_excel('CleanedPartA.xlsx')
```

Figure 4: Load excel file into PyCharm

This line of code shown in Figure is to load the Excel file that has been done in advance to the python and store it into the variable name AR\_data.

## Step 4: Verifying the loaded dataset

```
print("Sample of dataset:\n", AR_data.head(10))
print("Information of dataset:\n", AR_data.info())
```

Figure 5: Display of Dataset

The verification of the dataset is required to identify the detailed information regarding the current loaded dataset, and hence, make an informed decision on how to process these data.

Output (Refer to [Appendix A](#))

Since the loaded dataset only processes until the cleaning part, a further encoding method is necessary in order to apply the Apriori Algorithm.

### Step 5: Define a drug-type list

```
drug_col = ['Heroin', 'Heroin_DC', 'Cocaine', 'Fentanyl', 'Ethanol', 'Fent_analogue', 'Oxycodone',
            'Oxymorphone', 'Hydrocodone', 'Benzodiazepine', 'Morph', 'Methadone',
            'Meth/Amphetamine', 'Amphet', 'Tramad', 'Hydromorphone', 'Xylazine', 'Gabapentin',
            'Opiate_NOS', 'Heroin/Morph/Codeine', 'Any_opioid', 'Other_opioid']

print("number of item", len(drug_col))
```

Figure 6: drug type list definition

The “drug\_col” list is defined to store all of the features in relation to drug type in the dataset, this is because all of the drug types are represented in a binary form (Y/N), rendering them suitable targets for applying the association rule to the relationship between them and extract a meaningful insight. After that, the print statement will be used to output the total number of substances, which is, in this case, 22 substances.

### Step 6: Manually Encoding

```
AR_data[drug_col] = AR_data[drug_col].map(lambda x: x in ['Y', 'P'])
```

Figure 7: Process of Encoding

This line of code will manually encode the categorical data in drug\_col into a binary format (True/False), the value ‘Y’ and ‘P’ will be encoded to True and the rest, which is ‘N’, will be encoded into False.

After Encoding:

|   | Heroin | Heroin_DC | Cocaine | Fentanyl | Ethanol | Fent_analogue | Oxycodone | Oxymorphone | Hydrocodone | Benzodiazepine | Morph |
|---|--------|-----------|---------|----------|---------|---------------|-----------|-------------|-------------|----------------|-------|
| 0 | False  | False     | True    | False    | False   | False         | False     | False       | False       | False          | False |
| 1 | True   | False     | False   | False    | False   | False         | False     | False       | False       | False          | False |
| 2 | True   | False     | False   | False    | False   | False         | False     | False       | False       | False          | False |
| 3 | True   | False     | False   | False    | False   | False         | False     | False       | False       | False          | False |
| 4 | False  | False     | False   | True     | False   | False         | False     | False       | False       | False          | False |
| 5 | False  | False     | True    | False    | False   | False         | False     | False       | False       | False          | False |
| 6 | False  | False     | False   | False    | False   | False         | False     | False       | False       | False          | False |
| 7 | False  | False     | False   | False    | False   | False         | True      | False       | False       | False          | False |
| 8 | False  | False     | True    | False    | False   | False         | False     | False       | False       | False          | False |
| 9 | True   | False     | False   | False    | False   | False         | False     | False       | False       | False          | False |

Figure 8: After applying the encoding method to drug\_col list

## 1.2 Single-Level Association Rule

### Step 1: Define an empty list

```
substances = []
```

Figure 9: substances list definition

Initialise an empty list, and name it substances to store the drug type that each individual uses in the record.

### Step 2: Store drug type usage for each individual (include 0 and 1 substances)

```
for i in range(len(AR_data)):
    substance = [drug for drug in drug_col if AR_data.at[i, drug]]
    if substance:
        substances.append(substance)
```

Figure 10: Storing drug type usage for rule generation

In this step, the substances used by an individual will be stored in the predefined list by iterating the dataset, if the drug\_col is present (True), it will be appended to the list. If the person did not use any drug, that entry will be skipped

### Step 3: Verifying the Appended Result

```
print(f"\nTotal Substances: {len(substances)}")
print(substances[:10])
```

Figure 11: verify the items in substances list

The print statement above will count the number of the elements contained in the substances list, the first 10 elements in the list will also be printed.

Output:

```
Total Substances: 11880
[['Cocaine'], ['Heroin'], ['Heroin'], ['Heroin'], ['Fentanyl', 'Any_opioid'], ['Cocaine'], ['Any_opioid'], ['Oxycodone'], ['Cocaine'],
['Heroin']]
```

Figure 12: first 10 itemset in the list

Based on the output, the total number of substances contained in the list is 11880, including those with only one substance.

### Step 4: Identify the people with only one or no substance used

```
idx = []
for i in range(len(substances)):
    if len(substances[i]) < 2:
        idx.append(i)
```

Figure 13: individual with only one type of drug use

Define an empty list, name it `idx`, and iterate through the “substances” list to store the people with 1 or 0 substances used in “`idx`” list.

#### Step 5: Remove the row with 1 or 0 substances

```
RF_drug = AR_data.copy()
RF_drug.drop(idx, axis=0, inplace=True)
RF_drug.reset_index(drop=True, inplace=True)

print("\nTable after removing the entries that contain one or zero:\n", RF_drug[drug_col].head(10))
```

Figure 14: Process of removing the row

Make a copy of the previous code and rename it as `RF_drug`, indicating refined features. The people with less than 2 substances used will then be removed from the dataset. After that, the index will be reset. The first 10 rows will be printed to the console to verify that the 1 or 0 substances are properly removed.

Output (Refer to [Appendix B](#)):

According to the output, all of the unique or no substances have been removed from the datasets.

#### Step 6: Redo step 2

```
s = []
for i in range(len(RF_drug)):
    sub = [drug for drug in drug_col if RF_drug.at[i, drug]]
    if sub:
        s.append(sub)

print(s[:5])
print("Total substances", len(s))
```

Figure 15: store the drug type after removing action

A new list is created to store the substances used that contain at least 2. The first 5 substances and the total substances will be printed to console to verify that the unnecessary entries are properly removed.

Output:

```
[['Fentanyl', 'Any_opioid'], ['Heroin', 'Heroin_DC', 'Heroin/Morph/Codeine', 'Any_opioid'], ['Heroin', 'Heroin_DC', 'Heroin/Morph/Codeine', 'Any_opioid'], ['Heroin', 'Heroin_DC', 'Cocaine', 'Heroin/Morph/Codeine', 'Any_opioid'], ['Heroin', 'Ethanol']]
Total substances 10653
```

Figure 16: first 5 items after removing action

According to the output, the rows with only one substance used have been eliminated, and the total number of substances in the list has also been reduced to 10653.

## Step 7: Determine the threshold for support

```

77 num_rules = []
78 confidence_level = [0.2, 0.4, 0.6, 0.8, 1]
79 for conf in confidence_level:
80     ar_c = list(ap(s, min_support=0.001, min_confidence=conf, min_lift=3.5, min_length=2))
81     # ar_c = list(ap(s, min_support=0.005, min_confidence=conf, min_lift=3.5, min_length=2))
82     # ar_c = list(ap(s, min_support=0.01, min_confidence=conf, min_lift=3.5, min_length=2))
83     # ar_c = list(ap(s, min_support=0.05, min_confidence=conf, min_lift=3.5, min_length=2))
84     num_rules.append(len(ar_c))
85
86 plt.figure(figsize=(10, 6))
87 plt.plot(*args: confidence_level, num_rules, marker='o', linestyle='-', color='black')
88 plt.xlabel("Confidence")
89 plt.ylabel("Number of rules")
90 plt.title("Support Level (0.5%)")
91 plt.show()

```

Figure 17: identify the optimal support level threshold

These blocks of code will assist in determining the minimum support by visualising the number of rules based on different levels of confidence, including 0.2, 0.4, 0.6, 0.8 and 1.

Output:

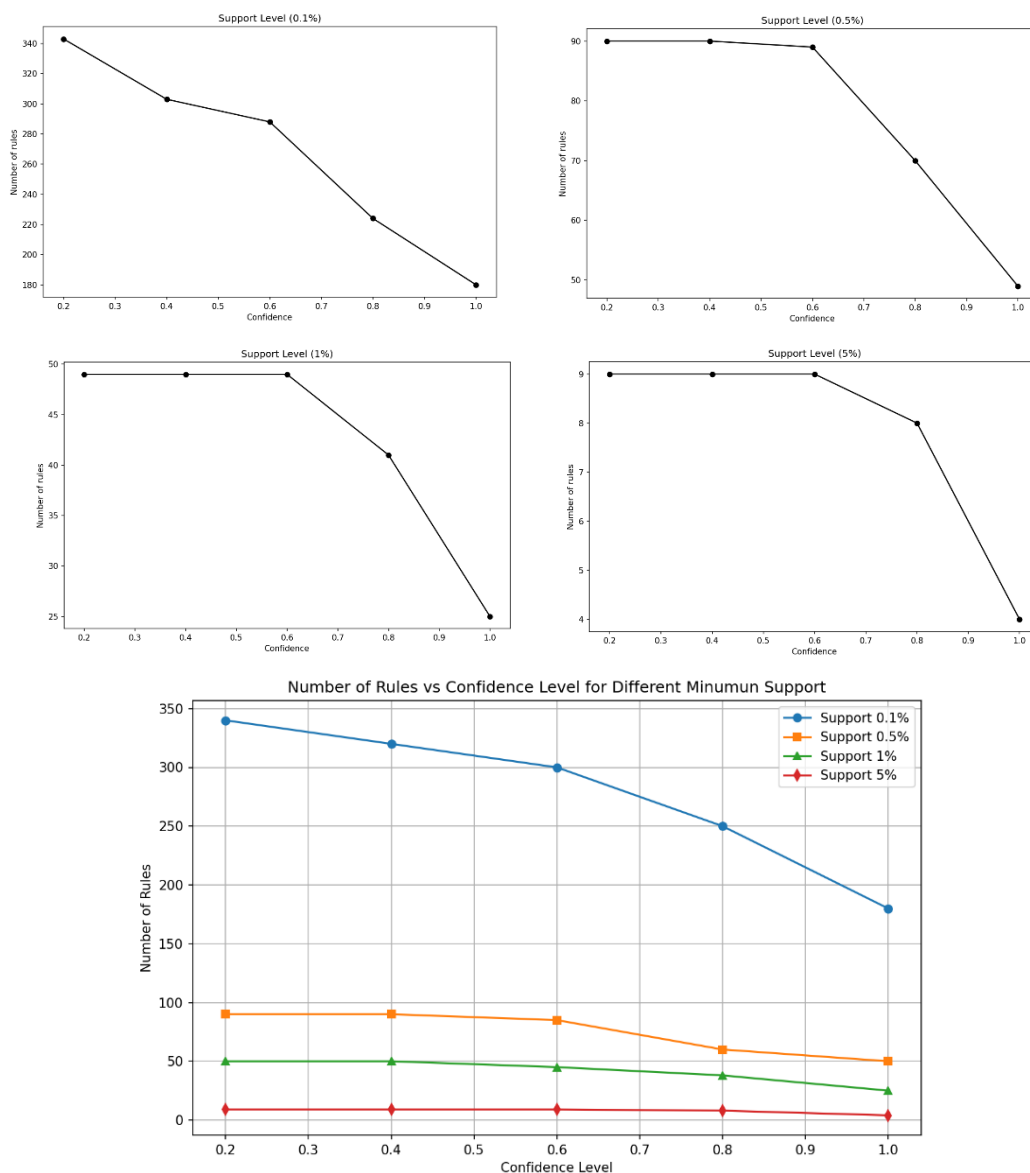


Figure 18: Comparison of support level value

According to the graph, the most appropriate support threshold can be determined by comparing the rule generated at different support levels.

- 0.01% Support Level. The rules generated at this level, which is about 350 of rules, are not many to be analysed and interpreted.
- 0.05% Support Level. The rules generated at this level are about 100 of rules, this is very useful if the goal is to reveal more patterns.
- 1% Support Level. There are approximately 50 rules generated at this level, which might be appropriate for a cleaner result.
- 5% Support Level. Only a few rules are generated at this level which might not be useful in identifying the association rules.

Furthermore, as the confidence increases, the lesser the rules will remain. This means that the unreliable rule can be discarded by increasing the confidence threshold. In this case, the 0.5% of minimum support and 6% of confidence thresholds will be chosen with a 3.5 lift to filter out more useful and reliable association rules.

### Step 8: Apply Apriori Algorithm

```
rd = dict({'Rule': [], 'Support': [], 'Confidence': [], 'Lift': []})
ar = ap(s, min_support=0.005, min_confidence=0.6, min_lift=3.5, min_length=2)
ar_results = list(ar)
```

Figure 19: Apriori Algorithm Application

The Apriori Algorithm will be applied in this step after determining the minimum threshold for support, confidence and lift. The result will then be stored in the list, and name it ar\_results to indicate the results of association rules. The dictionary, "rd" will be defined to store the necessary information including the rules and their support, confidence and lift.

### Step 9: Visualisation (Console)

```
for item in ar_results:
    pair = item[0]
    items = [x for x in pair]
    print("Rule: " + items[0] + " -> " + items[1])

    print("Support: " + str(item[1]))

    print("Confidence: " + str(item[2][0][2]))
    print("Lift: " + str(item[2][0][3]))
    print("-----")
    rd['Rule'].append(items[0] + " -> " + items[1])
    rd['Support'].append(item[1])
    rd['Confidence'].append(str(item[2][0][2]))
    rd['Lift'].append(str(item[2][0][3]))
```

Figure 20: Rules organisation in console

The code above is aimed at enhancing the organisation and readability of the rule results.

```
df = pd.DataFrame(rd)

df.sort_values(by=['Confidence', 'Lift'], ascending=[False, False], inplace=True)
df.drop_duplicates(subset=['Confidence', 'Lift'], keep='first', inplace=True)
df.reset_index(drop=True, inplace=True)

print("Association Rule:\n", df)
```

After formatting the association rules, the “rd” dictionary will be transformed into a data frame to enable a tabular representation. Rules that are redundant and share the same confidence and lift values will also be eliminated. The index will then be reset to ensure the index always start as 0. Finally, the result will be printed out to the console for analysis job.

| Association Rule: | Rule                                   | Support  | Confidence          | Lift               |
|-------------------|----------------------------------------|----------|---------------------|--------------------|
| 0                 | Any_opioid -> Heroin_DC                | 0.012954 | 1.0                 | 5.837260273972603  |
| 1                 | Heroin_DC -> Heroin                    | 0.059608 | 1.0                 | 5.872680525164114  |
| 2                 | Benzodiazepine -> Heroin_DC            | 0.012954 | 1.0                 | 5.82882304099637   |
| 3                 | Heroin_DC -> Heroin/Morph/Codeine      | 0.059608 | 1.0                 | 5.511122607346094  |
| 4                 | Benzodiazepine -> Heroin_DC            | 0.012954 | 1.0                 | 5.178901312591152  |
| 5                 | Heroin/Morph/Codeine -> Heroin_DC      | 0.059420 | 0.9968503937007874  | 5.8188752622435555 |
| 6                 | Heroin_DC -> Any_opioid                | 0.059420 | 0.9968503937007874  | 5.51087039133082   |
| 7                 | Heroin_DC -> Any_opioid                | 0.059420 | 0.9968503937007874  | 5.162589812394015  |
| 8                 | Any_opioid -> Heroin_DC                | 0.022984 | 0.9959183673469387  | 5.813434721833939  |
| 9                 | Fentanyl -> Heroin_DC                  | 0.022984 | 0.9959183673469387  | 5.050717804531083  |
| 10                | Fentanyl -> Heroin_DC                  | 0.022984 | 0.9959183673469387  | 5.157762039886698  |
| 11                | Any_opioid -> Heroin_DC                | 0.016521 | 0.9943502824858758  | 5.804281402368239  |
| 12                | Cocaine -> Heroin_DC                   | 0.016521 | 0.9943502824858758  | 5.4970490707431425 |
| 13                | Cocaine -> Heroin_DC                   | 0.016521 | 0.9943502824858758  | 5.149641983144485  |
| 14                | Any_opioid -> Ethanol                  | 0.012672 | 0.9854014598540146  | 5.103296913867194  |
| 15                | Cocaine -> Heroin/Morph/Codeine        | 0.056228 | 0.9771615008156607  | 5.060622979776185  |
| 16                | Heroin -> Heroin/Morph/Codeine         | 0.044776 | 0.954               | 5.490671852211959  |
| 17                | Heroin/Morph/Codeine -> Any_opioid     | 0.171313 | 0.9441282979824107  | 4.889547201675557  |
| 18                | Any_opioid -> Ethanol                  | 0.008873 | 0.924731182795689   | 4.7809091536374634 |
| 19                | Benzodiazepine -> Heroin/Morph/Codeine | 0.041866 | 0.908303054989817   | 4.704256589412505  |
| 20                | Oxycodone -> Oxymorphone               | 0.011358 | 0.8287671232076713  | 9.755644127058957  |
| 21                | Oxycodone -> Heroin/Morph/Codeine      | 0.007885 | 0.808762736923978   | 4.182958752477467  |
| 22                | Oxycodone -> Oxymorphone               | 0.005163 | 0.79771014492753623 | 9.382896949315397  |
| 23                | Oxycodone -> Oxymorphone               | 0.005163 | 0.79771014492753623 | 9.382896949315397  |
| 24                | Any_opioid -> Heroin                   | 0.006853 | 0.776595744680851   | 6.58684272936712   |
| 25                | Heroin -> Heroin/Morph/Codeine         | 0.006853 | 0.776595744680851   | 6.5711473138086623 |
| 26                | Any_opioid -> Heroin                   | 0.006853 | 0.776595744680851   | 4.293240512758223  |
| 27                | Benzodiazepine -> Fentanyl             | 0.006853 | 0.776595744680851   | 4.279914365279413  |
| 28                | Any_opioid -> Heroin                   | 0.022810 | 0.7689873417721519  | 6.522310630492623  |
| 29                | Fentanyl -> Fentanyl                   | 0.022810 | 0.7689873417721519  | 6.586768984828222  |
| 30                | Heroin/Morph/Codeine -> Fentanyl       | 0.022810 | 0.7689873417721519  | 4.251179113595687  |
| 31                | Fentanyl -> Heroin                     | 0.022810 | 0.7689873417721519  | 4.237983524003     |
| 32                | Any_opioid -> Heroin                   | 0.012109 | 0.7580235294117648  | 5.574997971602435  |
| 33                | Fentanyl -> Methadone                  | 0.012109 | 0.7580235294117648  | 4.518584158096292  |
| 34                | Any_opioid -> Heroin                   | 0.007228 | 0.7549019607843137  | 6.408284282502810  |
| 35                | Fentanyl -> Xylazine                   | 0.007228 | 0.7549019607843137  | 3.8758858524505913 |
| 36                | Heroin/Morph/Codeine -> Xylazine       | 0.007228 | 0.7549019607843137  | 4.173311511340393  |
| 37                | Xylazine -> Heroin                     | 0.007228 | 0.7549019607843137  | 4.16035726240832   |
| 38                | Any_opioid -> Heroin                   | 0.008261 | 0.7457627118644067  | 6.325326580666     |
| 39                | Heroin -> Heroin/Morph/Codeine         | 0.008261 | 0.7457627118644067  | 6.31025430400809   |
| 40                | Any_opioid -> Heroin                   | 0.008261 | 0.7457627118644067  | 4.122786803057538  |
| 41                | Cocaine -> Fentanyl                    | 0.008261 | 0.7457627118644067  | 4.109989741071661  |
| 42                | Any_opioid -> Ethanol                  | 0.006008 | 0.7272727272727274  | 6.16850028951939   |
| 43                | Ethanol -> Heroin                      | 0.006008 | 0.7272727272727274  | 6.1580817583563    |
| 44                | Any_opioid -> Ethanol                  | 0.006008 | 0.7272727272727274  | 4.020568948436     |
| 45                | Heroin/Morph/Codeine -> Fentanyl       | 0.006008 | 0.7272727272727274  | 4.00808916897897   |
| 46                | Any_opioid -> Heroin                   | 0.040176 | 0.698205546492659   | 5.12764392192158   |
| 47                | Cocaine -> Fentanyl                    | 0.040176 | 0.698205546492659   | 4.9157208422505    |

Based on the association rule mining result, there are approximately 68 rules that satisfy the

minimum threshold requirement, and from them, the most important association can be determined. First of all, there are a few rules with 100% of confidence that if a substance is used, the other will appear together as well, and these rules include:

- {Any\_opioid} → {Heroin\_DC}
- {Heroin\_DC} → {Heroin}
- {Benzodiazepine} → {Heroin\_DC}
- {Heroin\_DC} → Heroin/Morph/Codeine}

The 100% of confidence among them suggests that if an item that belongs to the antecedent is being used, the consequent will be used together as well. Besides that, these frequent items have a lift of 5, which ensures a strong association rule between them.

Furthermore, the rules {Oxymorphone} → {Oxycodone} has the highest lift (9.76) among all with around 83% of confidence. This means that this rule has the strongest association rule, and the antecedent and consequent are highly independent of each other.

Besides that, there are also some noticeable repeated rules with different matrix values such as confidence and lift.

### 1.3 Negative Association Rule

Typically, the association rule refers only to the positive association rule. However, the negative association rule such as the items that are not present in the substances list should be attached important to. This is because the negative association is beneficial in market basket analysis by pinpointing the substances that are conflicting with one another or that serve as a complement (Kotsiantis & Kanellopoulos, 2005). In this section, the negative association rule will be implemented using the algorithms that are proposed by Kavitha et al (2011) which are based on the Apriori Algorithm. This can be used to reveal the patterns where the occurrence of one substance reduces the likelihood of another.

#### **Step 1: Define a function to calculate the negative association rule**

```
def compute_neg_support(a, b, sub):
    # Calculate the positive support_a by using formula: number of A in itemset/length of the itemset
    supp_A = sum(a.issubset(set(i)) for i in sub) / len(sub)
    # Calculate the positive support_b by using formula: number of B in itemset/length of the itemset
    supp_B = sum(b.issubset(set(i)) for i in sub) / len(sub)
    # Calculate support(A->B) : count of A union B / length of the itemsets
    supp_AB = sum(a.union(b).issubset(set(i)) for i in sub) / len(sub)
    # Calculate the support(~A) by using the formula: 1 - support(A)
    supp_nA = 1 - supp_A
    # Calculate the support(~B) by using the formula: 1 - support(B)
    supp_nB = 1 - supp_B
    # Calculate the support(A union ~B): support(A) - support(A union B)
    supp_A_nB = supp_A - supp_AB
    # Calculate the support(~A union B): support(B) - support(B union A)
    supp_nA_B = supp_B - supp_AB

    return supp_A, supp_B, supp_AB, supp_nA, supp_nB, supp_A_nB, supp_nA_B
```

Figure 23: Function of negative support calculation

The function defined is to calculate the negative support, this step is necessary because the conventional way of support calculation can only identify the pattern where A and B occur together, but they are unable to identify the pattern in which A and B are mutually exclusive. First of all, declaring a few necessary parameters.

Table 1: Parameter description of compute\_neg\_support() function

| Parameter  | Parameter Description   |
|------------|-------------------------|
| <b>a</b>   | Represent the itemset A |
| <b>b</b>   | Represent the itemset B |
| <b>sub</b> | A list of the substance |

Secondly, performing computation by using the following formula,

- i. Calculation of Support A and B

$$\text{Support}(A) = \frac{\text{Number of Substance containing A}}{\text{Total Substances}}$$

$$\text{Support}(B) = \frac{\text{Number of Substance Containing B}}{\text{Total Substances}}$$

- ii. Calculation of Support (A U B)

$$\text{Support}(A \cup B) = \frac{\text{Number of Substance Containing both A and B}}{\text{Total Substances}}$$

- iii. Calculation of  $\text{Support}(\sim A)$  and  $\text{Support}(\sim B)$

$$\text{Support}(\sim A) = 1 - \text{Support}(A)$$

$$\text{Support}(\sim B) = 1 - \text{Support}(B)$$

- iv. Calculation of  $\text{Support}(\sim A \cup B)$  and  $\text{Support}(A \cup \sim B)$

$$\text{Support}(\sim A \cup B) = \text{Support}(B) - \text{Support}(B \cap A)$$

$$\text{Support}(A \cup \sim B) = \text{Support}(A) - \text{Support}(A \cap B)$$

The result of the calculation will then be returned for future use.

**Step 2: Define function for computing negative association rule** (Refer to [Appendix C](#))

### 1.) Define an empty list

```
neg_rules = []
```

Figure 24: *neg\_rules* list definition

Define a list that will store all the rules that are negatively associated

### 2.) Extraction of Item Pairs

```
for item in ar_results:
    pair = item.items
    items = list(pair)

    if len(items) < 2:
        continue

    A = {items[0]}
    B = {items[1]}
```

Figure 25: *item pair extraction*

Extract the item from “ar\_result” and store them inside the itemset A and B for further analysis job. If the number of items in the list is less than 2, then the items will be skipped.

### 3.) Function calling from step 1

```
(supp_A, supp_B, supp_AB, supp_nA, supp_nB, supp_A_nB,
supp_nA_B) = compute_neg_support(A, B, sub)
```

Figure 26: *Invoke the compute\_neg\_support() function*

Call the `compute_neg_support ()` function that have been created at step 1, and assigned the return value to `supp_A`, `supp_B`, `supp_nA`, `supp_nB`, `supp_A_nB` and `supp_nA_B`

### 4.) Negative Confidence Calculation

```
conf_A_nB = supp_A_nB / supp_A if supp_A else 0
conf_nA_B = supp_nA_B / supp_nA if supp_nA else 0
```

Figure 27: *calculation of negative confidence*

Compute the negative confidence by using the formula:

$$Confidence(A \rightarrow \sim B) = \left( \frac{support(A \cap \sim B)}{support(A)} \right)$$

$$Confidence(\sim A \rightarrow B) = \frac{support(\sim A \cap B)}{Support(\sim A)}$$

## 5.) Conviction Calculation

```
conf_A_nB = supp_A_nB / supp_A if supp_A else 0
conf_nA_B = supp_nA_B / supp_nA if supp_nA else 0
```

Figure 28: Conviction calculation

Conviction is defined as the ratio of the expected frequency that A occurs without B or vice versa (Kavitha et al., 2011). The formula can be expressed by the following:

$$Conv(A \rightarrow B) = \frac{1 - support(B)}{1 - confidence(A \rightarrow B)}$$

## 6.) Append the Negative Rules

```
if conf_A_nB >= min_confidence and supp_A_nB >= min_support:
    neg_rules.append(
        {'Rule': f"{A} → NOT_{B}",
         'Support': supp_A_nB,
         'Confidence': conf_A_nB,
         'Conviction': conv_A_nB
        })

if conf_nA_B >= min_confidence and supp_nA_B >= min_support:
    (neg_rules.append
     ({'Rule': f"NOT_{A} → {B}",
      'Support': supp_nA_B,
      'Confidence': conf_nA_B,
      'Conviction': conv_nA_B
      }))
)
```

Figure 29: Append egative rule

After defining the calculation, the rules which have met the condition such as minimum confidence and minimum support will be appended to the “neg\_rules” list. The list to be appended will include the rules as well as the metrics (Support, Confidence and Conviction).

## 7.) Return a data frame

```
return pd.DataFrame(neg_rules)
```

Figure 30: return the function

Finally, the entire function will be returned as a data frame.

### Step 3: Apply Negative Rules

```
negative_df = negative_rules(ar_results, substances, min_support: 0.005, min_confidence: 0.7)
```

Figure 31: Negative rule application

Apply the `negative_rules()` method that has been created previously and declare the `min_support` and `min_confidence`. The data frame return will then be stored in the variable, “`negative_df`”.

### Step 4: Display the result

```
negative_df.sort_values(by=["Confidence"], ascending=[False], inplace=True)
negative_df.drop_duplicates(subset=['Confidence', 'Conviction'], keep='first', inplace=True)
negative_df.reset_index(drop=True, inplace=True)
print("\nNegative Association Rules:\n", negative_df)
```

Figure 32: Display of negative rules

The result will be sorted in a descending order based on the confidence and the duplicated rules will also be removed. After removing the duplicated items, the index will be reset. The result will then be printed out to the console to conduct analysis tasks on the result.

Output:

| Negative Association Rules: |                                                   |          |            |            |
|-----------------------------|---------------------------------------------------|----------|------------|------------|
|                             | Rule                                              | Support  | Confidence | Conviction |
| 0                           | {'Any_opioid'} → NOT_{'Opiate_NOS'}               | 0.733754 | 0.987426   | 78.681321  |
| 1                           | {'Heroin'} → NOT_{'Methadone'}                    | 0.278451 | 0.924539   | 12.049369  |
| 2                           | {'Any_opioid'} → NOT_{'Oxycodone'}                | 0.686448 | 0.923765   | 11.995561  |
| 3                           | {'Heroin'} → NOT_{'Fent_analogue'}                | 0.271380 | 0.901062   | 9.278678   |
| 4                           | {'Oxycodone'} → NOT_{'Oxymorphone'}               | 0.074242 | 0.868110   | 7.478697   |
| 5                           | {'Cocaine'} → NOT_{'Heroin/Morph/Codeine'}        | 0.326936 | 0.848405   | 5.373849   |
| 6                           | {'Ethanol'} → NOT_{'Heroin/Morph/Codeine'}        | 0.222727 | 0.826617   | 4.698529   |
| 7                           | {'Fentanyl'} → NOT_{'Heroin/Morph/Codeine'}       | 0.558081 | 0.823705   | 4.620923   |
| 8                           | {'Heroin'} → NOT_{'Heroin_DC'}                    | 0.238889 | 0.793181   | 4.533957   |
| 9                           | {'Benzodiazepine'} → NOT_{'Heroin/Morph/Codeine'} | 0.181145 | 0.792050   | 3.917512   |
| 10                          | {'Any_opioid'} → NOT_{'Benzodiazepine'}           | 0.578030 | 0.777866   | 3.472210   |
| 11                          | {'Heroin'} → NOT_{'Benzodiazepine'}               | 0.232155 | 0.770822   | 3.365486   |
| 12                          | {'Any_opioid'} → NOT_{'Heroin/Morph/Codeine'}     | 0.558333 | 0.751359   | 3.276400   |
| 13                          | {'Heroin'} → NOT_{'Ethanol'}                      | 0.226094 | 0.750699   | 2.930412   |
| 14                          | {'Cocaine'} → NOT_{'Heroin'}                      | 0.289141 | 0.750328   | 2.798955   |
| 15                          | {'Any_opioid'} → NOT_{'Heroin'}                   | 0.545875 | 0.734594   | 2.633033   |
| 16                          | {'Any_opioid'} → NOT_{'Ethanol'}                  | 0.540657 | 0.727571   | 2.681640   |
| 17                          | NOT_{'Heroin'} → {'Fentanyl'}                     | 0.507492 | 0.726211   | 2.552405   |

Figure 33: Result of applying negative rule association

The result above has shown an interesting pattern of mutual exclusive substance, which means that the use of one substance may lead to less occurrence of another. For instance, an individual who uses “Any\_opioid”, will have a 98.74% confidence that they will not use “Opiate\_NOS” together due to the potential relationship among them such as they are a complement of each other, the strength of this negative rule has been proven by the high conviction.

## 1.4 Multi-Level Association Rule

In the real world, the sparsity of data has made the process of identifying the strong relationships among the primitive levels of abstraction difficult. Therefore, the multi-level association rule is needed to mine the association rules across various levels and discover a more meaningful pattern. In this section, the multi-level association rule mining will be implemented to uncover the relationship related to Accidental Drug Related Deaths in different dimensions.

### Step 1: Define an empty list

```
m_items = []
```

Figure 34: *m\_items* list definition

Define an empty list that will store the drug type and the features that are needed to discover an interesting empty.

### Step 2: Store the items in the list

```
for i in range(len(AR_data)):
    gender = [str(AR_data.at[i, demo]) for demo in ['Sex']]
    city = [str(AR_data.at[i, city]) for city in ['Res_city']]
    sub = [drug for drug in drug_col if AR_data.at[i, drug]]

    if gender and city and sub:
        m_items.append(gender + city + sub)
```

Figure 35: *Append the rules to list*

The features that are required for association rule mining, including the drug usage along with their gender and the place they reside, will be appended to the newly created list.

### Step 3: Verifying the List

```
print(f"Total Multi-Dimensional Itmes: {len(m_items)}")
print(m_items[:5])
```

Figure 36: *m\_items* list verification

The length of the list and the first 5 rows will be printed to the console to verify that the items are successfully adde

Output:

```
Total Multi-Dimensional Itmes: 11880
[['Male', 'STAMFORD', 'Cocaine'], ['Male', 'NORWICH', 'Heroin'], ['Male', 'HEBRON', 'Heroin'], ['Female', 'BALITIC', 'Heroin'], ['Male', 'SHELTON', 'Fentanyl', 'Any_opioid']]
```

Figure 37: *first 5 itemset in m\_items*

Based on the result, there are 11880 records contained in the list, and the first 5 elements in the list have also been printed to the console, which means that the elements that are

needed for the constructing association rule are successfully appended.

#### Step 4: Apply Apriori Algorithm

```
m_rd = {'Rule': [], 'Support': [], 'Confidence': [], 'Lift': []}
m_ar = ap(m_items, min_support=0.005, min_confidence=0.7, min_lift=1.5)

m_ar_results = list(m_ar)
```

Figure 38: Apriori Algorithm Application

Create a dictionary and name it “m\_rd” to store the rule and their respective matrices. After that, apply the Apriori Algorithm to the “m\_items” list with the following threshold:

- Minimum Support → 0.005
- Minimum Confidence → 0.7
- Minimum Lift → 1.5

By setting the minimum threshold of these matrices, a strong relationship can be identified to discover the important and meaningful insight.

#### Step 5: Visualisation (in console)

```
for item in m_ar_results:
    pair = item[0]
    items = list(pair)

    # Identify the antecedents and consequent
    antecedent = [x for x in items if x in AR_data['Sex'].values or x in AR_data['Res_city'].values]
    consequent = [x for x in items if x in drug_col]

    # Ensure at least one demographic + one city in antecedent & at least one drug in consequent
    if len(antecedent) >= 2 and len(consequent) >= 1:
        rule = f"({'', '.join(antecedent)}') --> {'', '.join(consequent)}"
        support = item[1]
        confidence = item[2][0][2]
        lift = item[2][0][3]

        m_rd['Rule'].append(rule)
        m_rd['Support'].append(support)
        m_rd['Confidence'].append(confidence)
        m_rd['Lift'].append(lift)
```

Figure 39: Code for organising the rules in console

These blocks of code help to formulate the items and their matrices in a cleaner and more readable way. The “Sex” and “Res\_city” columns will be stored in the list name antecedent, and the “drug\_col” will be stored in the list name consequent. The items will be represented in {antecedent} → {consequent} format.

## Step 6: Conversion to Data Frame

```
m_df = pd.DataFrame(m_rdl)
m_df.drop_duplicates(subset=['Confidence', 'Lift'], keep='first', inplace=True)
m_df.reset_index(drop=True, inplace=True)

print("\nFinal Rules:\n", m_df)
```

Figure 40: Convert the list to data frame

Convert the dictionary “m\_df” to DataFrame, drop the duplicated rules and reset the index. After that, print the result to the console to identify the association rule.

Output:

```
Final Rules:
      Rule      Support  Confidence  Lift
0  (MILFORD, Male) --> Any_opioid, Fentanyl 0.006734 0.851064 1.523375
1  (Male, BRIDGEPORT) --> Heroin/Morph/Codeine, Heroin 0.007744 0.707692 3.015561
2  (HARTFORD, Male) --> Fentanyl, Xylazine 0.013215 0.817708 1.563808
3  (HARTFORD, Male) --> Heroin/Morph/Codeine, Heroin 0.014141 0.788732 3.360883
4  (HARTFORD, Male) --> Heroin_DC, Heroin 0.005640 0.817073 3.481646
5  (HARTFORD, Male) --> Heroin/Morph/Codeine, Heroin_DC 0.005640 0.817073 5.646788
6  (NEW_HAVEN, Male) --> Heroin/Morph/Codeine, Heroin 0.008754 0.764706 3.258503
7  (Male, BRIDGEPORT) --> Heroin/Morph/Codeine, Heroin, Any_opioid 0.007744 0.707692 4.566749
8  (Male, HARTFORD) --> Heroin/Morph/Codeine, Fentanyl, Any_opioid 0.009848 0.847826 1.517579
9  (Male, HARTFORD) --> Fentanyl, Xylazine, Any_opioid 0.013215 0.817708 1.739992
10 (Male, HARTFORD) --> Heroin/Morph/Codeine, Heroin, Any_opioid 0.014141 0.788732 5.089702
11 (Male, HARTFORD) --> Heroin_DC, Heroin, Any_opioid 0.005640 0.817073 5.272585
12 (Male, HARTFORD) --> Heroin/Morph/Codeine, Heroin_DC, Any_opioid 0.005640 0.817073 5.663261
13 (NEW_HAVEN, Male) --> Heroin/Morph/Codeine, Heroin, Any_opioid 0.008670 0.757353 4.887210
14 (Male, WATERBURY) --> Heroin/Morph/Codeine, Heroin, Any_opioid 0.010943 0.764706 4.934658
15 (Male, BRIDGEPORT) --> Heroin/Morph/Codeine, Fentanyl, Heroin 0.005471 0.738636 3.147418
16 (Male, HARTFORD) --> Heroin/Morph/Codeine, Cocaine, Heroin 0.005892 0.760870 3.242156
17 (Male, HARTFORD) --> Heroin/Morph/Codeine, Fentanyl, Heroin 0.009848 0.847826 3.612688
18 (NEW_HAVEN, Male) --> Heroin/Morph/Codeine, Fentanyl, Heroin 0.005808 0.793103 3.379508
19 (Male, WATERBURY) --> Heroin/Morph/Codeine, Fentanyl, Heroin 0.008249 0.777778 3.314204
20 (Male, HARTFORD) --> Heroin/Morph/Codeine, Heroin_DC, Heroin 0.005640 0.817073 5.984397
21 (Male, BRIDGEPORT) --> Heroin/Morph/Codeine, Fentanyl, Heroin, Any_opioid 0.005471 0.738636 4.766431
22 (Male, HARTFORD) --> Fentanyl, Cocaine, Xylazine, Any_opioid 0.006987 0.783019 1.666177
23 (Male, HARTFORD) --> Heroin/Morph/Codeine, Cocaine, Heroin, Any_opioid 0.005892 0.760870 4.909902
24 (Male, HARTFORD) --> Heroin/Morph/Codeine, Fentanyl, Heroin, Any_opioid 0.009848 0.847826 5.471034
25 (NEW_HAVEN, Male) --> Heroin/Morph/Codeine, Fentanyl, Heroin, Any_opioid 0.005808 0.793103 5.117908
26 (Male, WATERBURY) --> Heroin/Morph/Codeine, Fentanyl, Heroin, Any_opioid 0.008249 0.777778 5.019011
27 (Male, HARTFORD) --> Heroin/Morph/Codeine, Heroin_DC, Heroin, Any_opioid 0.005640 0.817073 5.907991

Process finished with exit code 0
```

Figure 41: Result of Multi-Level Demensional Association

The Figure above has demonstrated the strong relationship between the information of the individual in the record (Sex, Residence City) and drug usage. According to the result, a number of interesting patterns can be revealed. For example, the rule {MILFORD, Male} → {Any\_opioid, Fentanyl} has a confidence of 85% with 1.5 lift, showing that if a person is a male and lives in MILFORD, he will have an 85% chance to use Fentanyl or Any\_opioid. Furthermore, the following rule {Male, BRIDGEPORT} → {Heroin/Morph/Codeine, Heroin} has also suggest that a male who resides in Bridgeport will have 70.77% confidence in using Heroin/Morph/Codeine and Heroin together with a lift of 3 indicating a strong association rule among them. The frequency of Fentanyl and Any opioid has suggested that they are the most common substance used by an individual on record.

## 2. Implementation of FP Algorithm

The FP-Growth, also known as Frequent Pattern Growth, is another method that can be utilised to uncover the frequent items in a large dataset. It was introduced by Han et al. (2000) to address the inefficiency in the Apriori algorithm by eliminating the necessity of generating a candidate itemset divide and conquer approach with a non-linear data structure, the FP-tree. Nowadays, the FP-growth has successfully been adopted in multiple domains including basket marker analysis and network traffic analysis (Lawal et al., 2014).

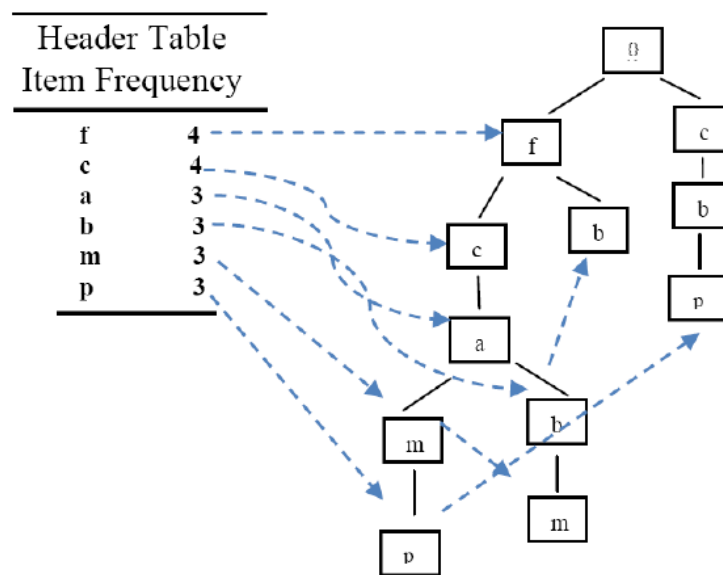


Figure 42: FP-growth Algorithm

In this section. The FP-growth algorithm will be applied to the same dataset in order to compare the performance between the Apriori algorithm and the FP-growth algorithm.

### 2.1 Preparation of FP-Growth

Similar to the Apriori Algorithm, some steps are required to be undertaken before the implementation of the algorithm.

#### Step 1: Importation of Packages

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
from mlxtend.frequent_patterns import association_rules as rulesmlx
from mlxtend.frequent_patterns import fpgrowth
from mlxtend.preprocessing import TransactionEncoder
import networkx as nx
```

Figure 43: Import libraries for FP-Growth

The pandas NumPy, matplotlib and networkx will function the same as Apriori Algorithm, which is to manipulate, calculate and visualise the data. Furthermore, the association\_rules from the mlxtend.frequent\_patterns library is used to extract the association rule from the frequent dataset. As the name suggests, the fpgrowth in the same package as association\_rules is used to perform the implementation of the FP-growth algorithm. Lastly, the TransactionEncoder from mlxtend.preprocessing is being imported to encode the categorical data into a binary form (True/False).

## Step 2: Set up PyCharm view (refer to Section 2.1, step 2)

## Step 3: Load the dataset

```
fp_data = pd.read_excel('CleanedPartA.xlsx')
```

Figure 44: Loading of Excel file to Python

Load the dataset into Python and name it as fp\_data to indicate that this dataset will be processed by using the FP-growth algorithm.

## Step 4: Define drug\_col column

```
drug_col = ['Heroin', 'Heroin_DC', 'Cocaine', 'Fentanyl', 'Ethanol', 'Fent_analogue', 'Oxycodone',
            'Oxymorphone', 'Hydrocodone', 'Benzodiazepine', 'Morph', 'Methadone', 'Meth/Amphetamine',
            'Amphet', 'Tramad', 'Hydromorphone', 'Xylazine', 'Gabapentin', 'Opiate_NOS',
            'Heroin/Morph/Codeine', 'Any_opioid', 'Other_opioid']
```

Figure 45: define drug\_col list for FP-growth

Define a list that will store the features that are to perform the association rule.

## Step 5: Perform Encoding

```
fp_data[drug_col] = fp_data[drug_col].map(lambda x: x == 'Y' or x == 'P')
```

Figure 46: Perform encoding on the features in drug\_col list

Perform manual encoding to the features in the drug\_col, this will manually encode the features into two forms, which the 'Y' and 'P' will be True to indicate the presence of substance, and the remaining 'N' will be false to indicate the absence of substance. The

encoding step is required because the mlxtend package only accepts the data in (0/1) pr (True/False) format.

Step 6: Verify the encoding result

```
print("New Encoded Dataset:\n", fp_data[drug_col].head(10))
print("\nSize of Dataset: ", fp_data.shape)
```

Figure 47: verify the encoding result

After Finishing the Encoding step, the dataset that contains the feature in the drug\_col will be displayed along with its size.

Output:

```
New Encoded Dataset:
  Heroin  Heroin_DC  Cocaine  Fentanyl  Ethanol  Fent_analogue  Oxycodone  Oxymorphone  Hydrocodone  Benzodiazepine  Morph  Methadone
0  False      False      True      False      False      False      False      False      False      False      False      False
1  True       False      False      False      False      False      False      False      False      False      False      False
2  True       False      False      False      False      False      False      False      False      False      False      False
3  True       False      False      False      False      False      False      False      False      False      False      False
4  False      False      False      True      False      False      False      False      False      False      False      False
5  False      False      True      False      False      False      False      False      False      False      False      False
6  False      False      False      False      False      False      False      False      False      False      False      False
7  False      False      False      False      False      False      True      False      False      False      False      False
8  False      False      True      False      False      False      False      False      False      False      False      False
9  True       False      False      False      False      False      False      False      False      False      False      False
```

Figure 48: Result of encoding

According to the sample result, the drug\_col has been encoded in an acceptable format for implementing the FP-growth algorithm.

## 2.2 Single-dimensional Association Rule

### Step 1: Identify the minimum support level

```
num_rules = []
confidence_level = [0.2, 0.4, 0.6, 0.8, 1]
for conf in confidence_level:
    fp_c = fpgrowth(fp_data[drug_col], min_support=0.005, use_colnames=True)
    # fp_c = fpgrowth(fp_data[drug_col], min_support=0.01, use_colnames=True)
    # fp_c = fpgrowth(fp_data[drug_col], min_support=0.05, use_colnames=True)
    # fp_c = fpgrowth(fp_data[drug_col], min_support=0.10, use_colnames=True)
    rule = rulesmlx(fp_c, metric='confidence', min_threshold=conf)
    enhance_r = rule[rule['lift'] > 1]
    num_rules.append(len(enhance_r))

plt.figure(figsize=(10, 6))
plt.plot(*args: confidence_level, num_rules, marker='o', linestyle='--', color='black')
plt.xlabel("Confidence")
plt.ylabel("Number of rules")
plt.title("Support Level (0.5%)")
plt.show()
```

Figure 49: Code for identify the support level threshold

The code above will help to determine the optimal minimum support level to select by evaluating how the number of rules has increased or decreased as the confidence level changes by using a for loop to loop through the confidence\_level list. The result will then be visualised individually.

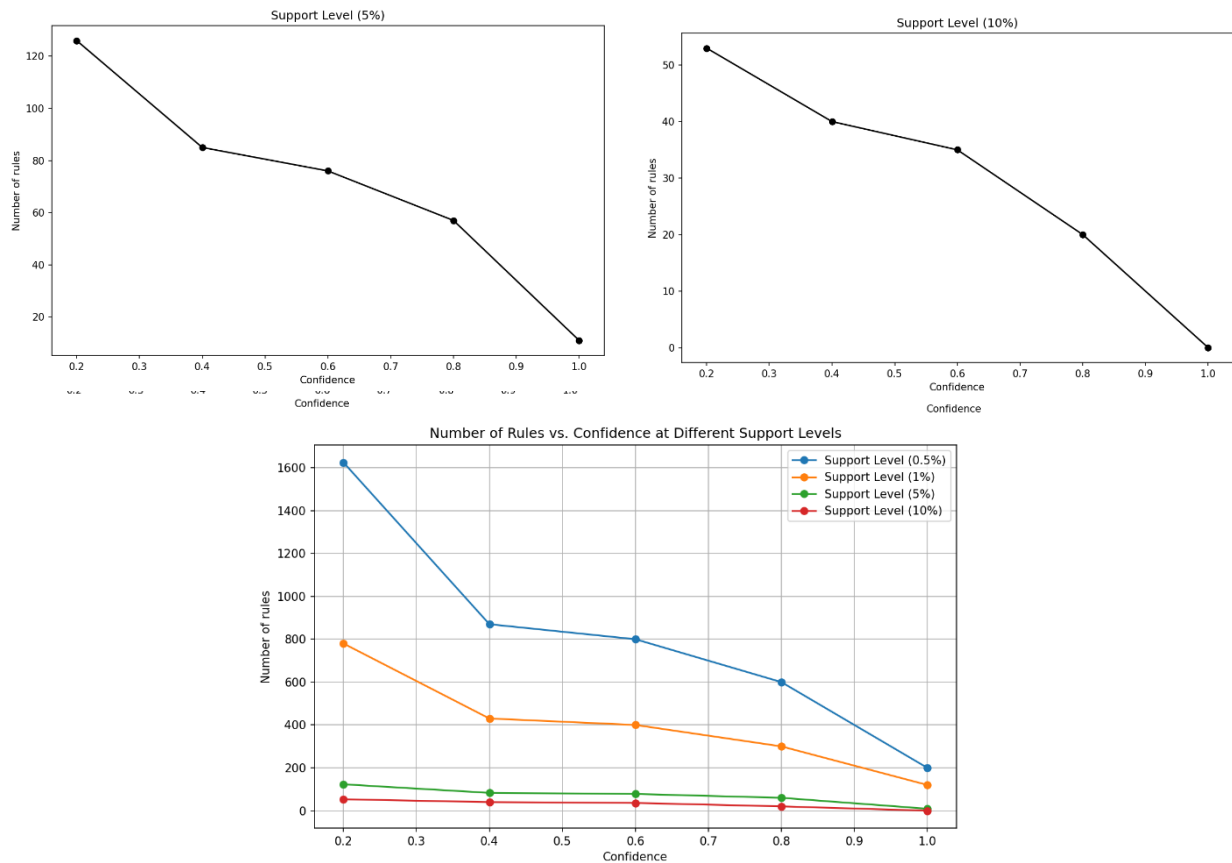


Figure 50: Comparison of minimum support level

Output:

The graph above has demonstrated the change in number of rules based on different support and confident level.

- 0.5% Support Level. In this level, there are about 1800 rules are being generated, which might not be efficient in analysing the relationship between frequent itemset.
- 1% Support Level. The number of rules generated at this level is still not considered efficient due to the high volume of rules is being generated (800 rules).
- 5% Support Level. This level has strike a good balance by generating a reasonable number of rules, which is around 120.
- 10% Support. This level has also generated a reasonable number of rules but might have overlooked some frequent patterns.

Based on the analysis of the graph, the 5% support level will be selected as a minimum threshold to uncover the important and meaningful association rule in order to produce an accurate analysis result.

## Step 2: Apply FP-Growth Algorithm

```
fp_res = fpgrowth(fp_data[drug_col], min_support=0.05, use_colnames=True)
```

Figure 51: Apply the association rule using fpgrowth

After determining the minimum support level, the FP-growth algorithm will be applied to the dataset that contains all the features in the “drug\_col” list. The use\_colnames will set to true to represent the actual name of the columns instead of the index.

## Step 3: Create different association rule

```
rules = rulesmlx(fp_res, metric="confidence", min_threshold=0.6)
```

Figure 52: generate different association rule using confidence as a metric

Another association rule will be applied to the frequent items by using rulesmlx, which is the association\_rules. Define the metrics, in this case, confidence, and set the minimum threshold to 0.6 to measure how often the rule has been proven to be true.

## Step 4: Strengthen the rule

```
enhanced_rules = rules[rules['lift'] > 1].copy()
enhanced_rules.sort_values(by=['confidence'], ascending=False, inplace=True)
enhanced_rules.reset_index(drop=True, inplace=True)
```

Figure 53: Further strengthen the rule by lift

The code above can help to strengthen the rules by filtering out the lift which is below 1 and storing the result of the copy in the variable “enhanced\_rules”. The copy will then be sorted in descending order based on the Confidence. The index will also be reset.

## Step 5: Display the result

```
print("\nOverall Results with FP-Growth:\n",
      enhanced_rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

Figure 54: Result of association rule by applying FP-growth

The result of the strengthened rule with their respective metrics will be displayed in the console.

Output (Refer to [Appendix D](#)):

Based on the output, the FP-growth has produced 76 of the association rules which has reached the minimum threshold. This association rule has been arranged in a way that the

most confident rules will appear at the top. This can help to identify the strongest rule in an efficient manner.

**Table 2: The frequent items with confidence of 1**

| Rule                                                     | Support  | Confidence | Lift     |
|----------------------------------------------------------|----------|------------|----------|
| {Heroin_DC, Any_opioid} → {Heroin/Morph/Codeine}         | 0.061598 | 1.000000   | 5.440963 |
| {Heroin_DC, Any_opioid, Heroin} → {Heroin/Morph/Codeine} | 0.061598 | 1.000000   | 5.440963 |
| {Heroin_DC, Any_opioid} → {Heroin/Morph/Codeine, Heroin} | 0.061598 | 1.000000   | 5.751800 |
| {Fentanyl, Xylazine} → {Any_opioid}                      | 0.089642 | 1.000000   | 1.357159 |
| {Heroin/Morph/Codeine, Heroin_DC, Any_opioid} → {Heroin} | 0.061598 | 1.000000   | 3.348519 |
| {Heroin_DC} → {Heroin/Morph/Codeine}                     | 0.061764 | 1.000000   | 5.440963 |
| {Heroin_DC, Heroin} → {Heroin/Morph/Codeine}             | 0.061764 | 1.000000   | 5.440963 |
| {Heroin_DC} → {Heroin}                                   | 0.061764 | 1.000000   | 3.348519 |
| {Heroin/Morph/Codeine, Heroin_DC} → {Heroin}             | 0.061764 | 1.000000   | 3.348519 |
| {Heroin_DC} → {Heroin/Morph/Codeine, Heroin}             | 0.061764 | 1.000000   | 5.751800 |
| {Heroin_DC, Any_opioid} → {Heroin}                       | 0.061598 | 1.000000   | 3.348519 |

The rules above have shown a strong relationship between them where the antecedents occur, the other will have 100% of confidence and will occur together as well. For instance, there is 8.96% that the rule {Fentanyl, Xylazine} → {Any\_opioid} will appear together. Every time the Fentanyl and Xylazine appear together, there is 100% certainty that the Any\_opioid will be used together, and the lift greater than 1 has shown the strength of the association rule.

Furthermore, the result has also suggested that the occurrence of the itemset, fentanyl and Any\_opioid is 60.43%, which is the highest support among all. There is also showing 82.01% confidence that if fentanyl has been used, the Any\_opioid will be used together as well. The lift with 1.22 suggest a strong relationship of the rule.

## 2.3 Multi-Level Association Rule

### Step 1: Discrete the age into age group

```
bins = [0, 12, 20, 30, 50, 100]
labels = ['Child', 'Adolescent', 'Young-Adult', 'Adult', 'Senior']

fp_data["Age_Group"] = pd.cut(fp_data["Age"], bins=bins, labels=labels, right=True)
```

Figure 55: Discretisation of age

The code above will assist in discretising the age into an interval range and maps them with its corresponding labels, where:

- 0 – 12 → Child
- 13 – 20 → Adolescent
- 21 – 30 → Young-Adult
- 31 – 50 → Adult
- 51 – 100 → Senior

The discretisation of age into age groups can help in identifying the multi-level association efficiently.

## Step 2: Encode the antecedents

```
te = TransactionEncoder()
df_out = fp_data[['Age_Group', 'Res_city']].astype(str).values.tolist()
te_ary = te.fit(df_out).transform(df_out)
df_encoded = pd.DataFrame(te_ary, columns=te.columns_)
```

Figure 56: Apply Transaction Encoding method

The Figure illustrates the Transaction Encoding that is applied to the “Age\_Group” and “Res\_city” features, which will be utilised to identify the association of the rules in different dimensions. The encoded features will then be stored as a data frame and assigned with a name, “df\_encoded”.

## Step 3: Perform Concatenation

```
df_final = pd.concat( objs: [df_encoded, fp_data[drug_col]], axis=1)
```

Figure 57: Process of concatenation

The “df\_encoded” will concatenate with the features inside the “drug\_col” list.

## Step 4: Verify the result

```
print("\nResult after Apply Transaction Encoding and Concatination:\n", df_final.head())
```

Figure 58: Code for Result verification

Verify the encoding and concatenation result by displaying the first 5 rows inside the “df\_encoded” data frame in the console.

Output:

Output:

| Result after Apply Transaction Encoding and Concatination: |         |       |                |       |         |        |        |         |           |         |                   |         |         |       |
|------------------------------------------------------------|---------|-------|----------------|-------|---------|--------|--------|---------|-----------|---------|-------------------|---------|---------|-------|
|                                                            | ADAMANT | AKRON | ALFRED_STATION | ALLEN | ALLSTON | AMENIA | AMSTON | ANDOVER | ANN_ARBOR | ANSONIA | ARLINGTON_HEIGHTS | ASHFORD | ASTORIA | ATHOL |
| 0                                                          | False   | False | False          | False | False   | False  | False  | False   | False     | False   | False             | False   | False   | False |
| 1                                                          | False   | False | False          | False | False   | False  | False  | False   | False     | False   | False             | False   | False   | False |
| 2                                                          | False   | False | False          | False | False   | False  | False  | False   | False     | False   | False             | False   | False   | False |
| 3                                                          | False   | False | False          | False | False   | False  | False  | False   | False     | False   | False             | False   | False   | False |
| 4                                                          | False   | False | False          | False | False   | False  | False  | False   | False     | False   | False             | False   | False   | False |

|   | DANBURY | DANIELSON | DARIEN | DAVIE | DAYVILLE | DEEP_RIVER | DEER_LODGE | DELAND |
|---|---------|-----------|--------|-------|----------|------------|------------|--------|
| 0 | False   | False     | False  | False | False    | False      | False      | False  |
| 1 | False   | False     | False  | False | False    | False      | False      | False  |
| 2 | False   | False     | False  | False | False    | False      | False      | False  |
| 3 | False   | False     | False  | False | False    | False      | False      | False  |
| 4 | False   | False     | False  | False | False    | False      | False      | False  |

|   | TAFTVILLE | TALLAHASSEE | TAMARAC | TARIFFVILLE | TAUNTON | TERRYVILLE | THOMASTON | THOMPSON | TOLLAND | TORRINGTON | TRUMBULL | UNCAVILLE | UNION | WRIGHTS | Young-Adult | ZIONSVILLE | Heroin | Heroin_DC | Cocaine | Fentanyl | Ethanol |
|---|-----------|-------------|---------|-------------|---------|------------|-----------|----------|---------|------------|----------|-----------|-------|---------|-------------|------------|--------|-----------|---------|----------|---------|
| 0 | False     | False       | False   | False       | False   | False      | False     | False    | False   | False      | False    | False     | False | False   | False       | False      | False  | False     | False   | False    | False   |
| 1 | False     | False       | False   | False       | False   | False      | False     | False    | False   | False      | False    | False     | False | False   | False       | False      | True   | False     | False   | False    | False   |
| 2 | False     | False       | False   | False       | False   | False      | False     | False    | False   | False      | False    | False     | False | False   | True        | False      | True   | False     | False   | False    | False   |
| 3 | False     | False       | False   | False       | False   | False      | False     | False    | False   | False      | False    | False     | False | False   | True        | False      | True   | False     | False   | False    | False   |
| 4 | False     | False       | False   | False       | False   | False      | False     | False    | False   | False      | False    | False     | False | False   | True        | False      | True   | False     | False   | False    | False   |

Figure 59: Result of encoding and concatenation

According to the sample above, the “Age\_Group” and “Res\_city” has been successfully encoded and concatenated with the “drug\_col” features and renamed to “df\_final”.

### Step 5: Apply FP-growth

```
Step 5: Apply FP-growth
frequent_itemsets = fpgrowth(df_final, min_support=0.005, use_colnames=True)
```

Figure 60: Apply fpgrowth algorithm

The fpgrowth() function will be applied to “df\_final” and set the min\_support to 0.005 to obtain the most valuable and meaningful association patterns. The column name will set to be true to display the actual column name instead of the index position. The result will be assigned to the variable, frequent\_itemsets.

### Step 6: Generate Association Rule

```
rules = rulesmlx(frequent_itemsets, metric="confidence", min_threshold=0.6)
```

Figure 61: Generate Association Rule by FP-growth

Generate the association rule and apply it to frequent\_itemsets and assign with a minimum of 0.6 confidence.

### Step 7: Filter the Rule

```
age_city_cols = [col for col in df_encoded.columns]
fp_result = rules[
    rules['antecedents'].apply(lambda x: len(x) == 2 and all(item in age_city_cols for item in x)) &
    rules['consequents'].apply(lambda x: any(item in drug_col for item in x))
]
```

Figure 62: rule filtration

The code above is to ensure that only the “Age\_Group” and “Res\_city” can show in the antecedents’ side, and the consequents will always be the drug type such as Heroin, Fentanyl and Cocaine.

### Step 8: Strengthen the rule

```
fp_result = fp_result[fp_result['lift'] >= 1]
fp_result.sort_values(by=['confidence'], ascending=[False], inplace=True)
fp_result.reset_index(drop=True, inplace=True)
```

Figure 63: strengthen the rule by lift

The strengths of the rule can be ensured by setting the limit of lift to 1 to filter out the weak association rule. After that, the result will be sorted in a descending order by confidence index will be reset to ensure the consistent position of each rule.

### Step 9: Display the overall result

```
print("\n Result of Multi-level Association Rule (FP-Growth)\n",
      fp_result[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

Figure 64: Result of multi-level dimensional association by applying FP-growth

Finally, the result of the multi-level association rule for FP-growth will be displayed.

Output:

| Result of Multi-level Association Rule (FP-Growth) |                          |                        |          |            |          |
|----------------------------------------------------|--------------------------|------------------------|----------|------------|----------|
|                                                    | antecedents              | consequents            | support  | confidence | lift     |
| 0                                                  | (NEW_BRITAIN, Senior)    | (Any_opioid)           | 0.010767 | 0.843137   | 1.144271 |
| 1                                                  | (HARTFORD, Adult)        | (Any_opioid)           | 0.047659 | 0.812233   | 1.102330 |
| 2                                                  | (Senior, WEST_HAVEN)     | (Any_opioid)           | 0.007345 | 0.807339   | 1.095688 |
| 3                                                  | (MIDDLETOWN, Senior)     | (Any_opioid)           | 0.005175 | 0.805195   | 1.092777 |
| 4                                                  | (Young-Adult, HARTFORD)  | (Any_opioid)           | 0.011769 | 0.801136   | 1.087269 |
| 5                                                  | (Young-Adult, WATERBURY) | (Any_opioid)           | 0.005926 | 0.797753   | 1.082677 |
| 6                                                  | (BRISTOL, Adult)         | (Any_opioid)           | 0.009599 | 0.793103   | 1.076368 |
| 7                                                  | (BRISTOL, Senior)        | (Any_opioid)           | 0.007261 | 0.790909   | 1.073389 |
| 8                                                  | (HARTFORD, Adult)        | (Fentanyl)             | 0.046407 | 0.790896   | 1.177255 |
| 9                                                  | (NEW_HAVEN, Senior)      | (Any_opioid)           | 0.020115 | 0.790164   | 1.072378 |
| 10                                                 | (Young-Adult, HARTFORD)  | (Fentanyl)             | 0.011518 | 0.784091   | 1.167126 |
| 11                                                 | (Senior, TORRINGTON)     | (Any_opioid)           | 0.005808 | 0.779221   | 1.057527 |
| 12                                                 | (WATERBURY, Senior)      | (Any_opioid)           | 0.016693 | 0.778210   | 1.056155 |
| 13                                                 | (HARTFORD, Senior)       | (Any_opioid)           | 0.034722 | 0.776119   | 1.053317 |
| 14                                                 | (EAST_HARTFORD, Senior)  | (Any_opioid)           | 0.005509 | 0.767442   | 1.041541 |
| 15                                                 | (Young-Adult, WATERBURY) | (Fentanyl)             | 0.005676 | 0.764045   | 1.137287 |
| 16                                                 | (EAST_HARTFORD, Adult)   | (Fentanyl)             | 0.005509 | 0.758621   | 1.129213 |
| 17                                                 | (BRIDGEPORT, Adult)      | (Any_opioid)           | 0.019281 | 0.757377   | 1.027881 |
| 18                                                 | (WATERBURY, Adult)       | (Any_opioid)           | 0.023370 | 0.756757   | 1.027039 |
| 19                                                 | (NEW_LONDON, Adult)      | (Any_opioid)           | 0.006427 | 0.754902   | 1.024522 |
| 20                                                 | (MANCHESTER, Adult)      | (Any_opioid)           | 0.006677 | 0.754717   | 1.024271 |
| 21                                                 | (BRIDGEPORT, Adult)      | (Fentanyl)             | 0.019197 | 0.754098   | 1.122481 |
| 22                                                 | (NEW_HAVEN, Adult)       | (Any_opioid)           | 0.021534 | 0.745665   | 1.011986 |
| 23                                                 | (MANCHESTER, Adult)      | (Fentanyl)             | 0.006594 | 0.745283   | 1.109360 |
| 24                                                 | (BRIDGEPORT, Senior)     | (Any_opioid)           | 0.016359 | 0.745247   | 1.011419 |
| 25                                                 | (TORRINGTON, Adult)      | (Any_opioid)           | 0.006093 | 0.744898   | 1.010945 |
| 26                                                 | (MILFORD, Adult)         | (Any_opioid)           | 0.005091 | 0.743902   | 1.009594 |
| 27                                                 | (HARTFORD, Senior)       | (Fentanyl)             | 0.033136 | 0.740672   | 1.102496 |
| 28                                                 | (WEST_HAVEN, Adult)      | (Any_opioid)           | 0.006176 | 0.740000   | 1.004298 |
| 29                                                 | (HARTFORD, Adult)        | (Fentanyl, Any_opioid) | 0.042734 | 0.728307   | 1.205228 |
| 30                                                 | (EAST_HARTFORD, Senior)  | (Fentanyl)             | 0.005175 | 0.720930   | 1.073110 |
| 31                                                 | (MIDDLETOWN, Adult)      | (Fentanyl)             | 0.005425 | 0.714286   | 1.063220 |
| 32                                                 | (NORWICH, Adult)         | (Fentanyl)             | 0.007011 | 0.711864   | 1.059616 |
| 33                                                 | (BRISTOL, Senior)        | (Fentanyl)             | 0.006510 | 0.709091   | 1.055487 |
| 34                                                 | (Young-Adult, HARTFORD)  | (Fentanyl, Any_opioid) | 0.010350 | 0.704545   | 1.165906 |
| 35                                                 | (BRISTOL, Adult)         | (Fentanyl)             | 0.008513 | 0.703448   | 1.047088 |
| 36                                                 | (NEW_BRITAIN, Adult)     | (Fentanyl)             | 0.012436 | 0.696262   | 1.036391 |
| 37                                                 | (NEW_BRITAIN, Senior)    | (Fentanyl)             | 0.008847 | 0.692810   | 1.031254 |
| 38                                                 | (HARTFORD, Senior)       | (Fentanyl, Any_opioid) | 0.030882 | 0.690299   | 1.142330 |
| 39                                                 | (WEST_HAVEN, Adult)      | (Fentanyl)             | 0.005759 | 0.690000   | 1.027070 |
| 40                                                 | (WATERBURY, Senior)      | (Fentanyl)             | 0.014690 | 0.684825   | 1.019367 |
| 41                                                 | (Young-Adult, WATERBURY) | (Fentanyl, Any_opioid) | 0.005008 | 0.674157   | 1.115619 |
| 42                                                 | (TORRINGTON, Adult)      | (Fentanyl)             | 0.005509 | 0.673469   | 1.002464 |
| 43                                                 | (NEW_HAVEN, Adult)       | (Fentanyl)             | 0.019447 | 0.673410   | 1.002377 |
| 44                                                 | (BRISTOL, Senior)        | (Fentanyl, Any_opioid) | 0.006176 | 0.672727   | 1.113252 |
| 45                                                 | (MANCHESTER, Adult)      | (Fentanyl, Any_opioid) | 0.005926 | 0.669811   | 1.108427 |
| 46                                                 | (NEW_BRITAIN, Senior)    | (Fentanyl, Any_opioid) | 0.008513 | 0.666667   | 1.103223 |
| 47                                                 | (WATERBURY, Senior)      | (Fentanyl, Any_opioid) | 0.014189 | 0.661479   | 1.094637 |
| 48                                                 | (BRIDGEPORT, Adult)      | (Fentanyl, Any_opioid) | 0.016610 | 0.652459   | 1.079712 |
| 49                                                 | (BRISTOL, Adult)         | (Fentanyl, Any_opioid) | 0.007846 | 0.648276   | 1.072789 |
| 50                                                 | (NEW_LONDON, Adult)      | (Fentanyl, Any_opioid) | 0.005509 | 0.647059   | 1.070775 |
| 51                                                 | (NEW_HAVEN, Senior)      | (Fentanyl, Any_opioid) | 0.016276 | 0.639344   | 1.058009 |
| 52                                                 | (WATERBURY, Adult)       | (Fentanyl, Any_opioid) | 0.019614 | 0.635135   | 1.051043 |
| 53                                                 | (NEW_HAVEN, Adult)       | (Fentanyl, Any_opioid) | 0.018279 | 0.632948   | 1.047424 |
| 54                                                 | (TORRINGTON, Adult)      | (Fentanyl, Any_opioid) | 0.005175 | 0.632653   | 1.046936 |

Figure 65: Result of multi-dimensional association rules

The figure above shows the result after applying the FP-growth algorithm, which contains 55 rules. Where:

- Antecedents → Contain the “Age\_Group” and the “Res\_city” of the individual in the record.
- Consequents → the drug type (e.g. Heroin, Fentanyl) that is associated with given antecedents
- Support → the frequency of itemset appears together
- Confidence → the likelihood of one item occurring that resulted in the occurrence of the other
- Lift → The strength of the association rule

According to the result, a Senior who lives in New Britain has an 84% of confidence that will

use Any\_opioid, with a lift of 1.14 to ensure the strength of the rule. Furthermore, the rules  $\{\text{HARTFORD, Adult}\} \rightarrow \{\text{Any\_opioid}\}$  is about 4% and with a confidence of 0.812233. This means that an adult who resides in Hartford will have an 81.22% probability of opioid use.

Furthermore, according to the result, Any\_opioid is the most common drug that is used by an individual, followed by Fentanyl, and some cities (e.g. Hartford) have appeared frequently in antecedents, showing high drug usage in these locations. The frequent appearance of adults in the antecedent also suggests that this age group is highly addicted to drugs.

### 3. Summary of Observations

#### 3.1 Key Finding

Upon implementing both frequent-use algorithms, which are the Apriori Algorithm and the FP-Growth Algorithm in the selected dataset, several implications were able to be identified. First of all, most of the rules fall below a fairly low support threshold, normally between 0.5% and 2%. This indicates that more interesting rules can be revealed within this range. The FP-growth algorithm has also discovered a more frequent itemset with a higher support level compared to Apriori Algorithm.

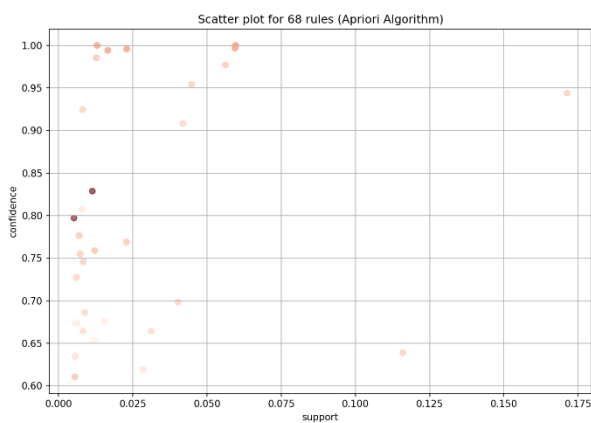


Figure 66: Scatter plot for Apriori Algorithm

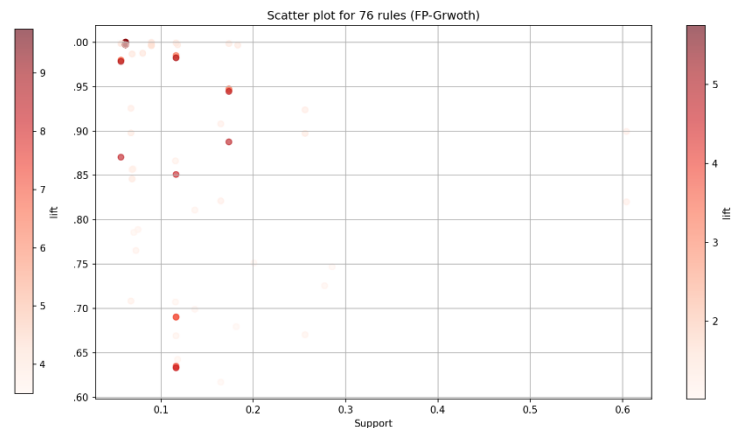


Figure 67: Scatter plot for FP-growth algorithm

Furthermore, The FP-growth can detect more robust association rules compared to the Apriori Algorithm which only generates a few strong rules. The rules generated from both algorithms may differ slightly but share similar patterns, as the result from both algorithms suggested that the opioid type of drugs such as “Heroin”, “Heroin\_DC”, “Heroin/Morph/Cocaine” and “Any\_opioid” are frequently used in conjunction with other substances. This indicates the combination of these drug types might lead to higher risk.

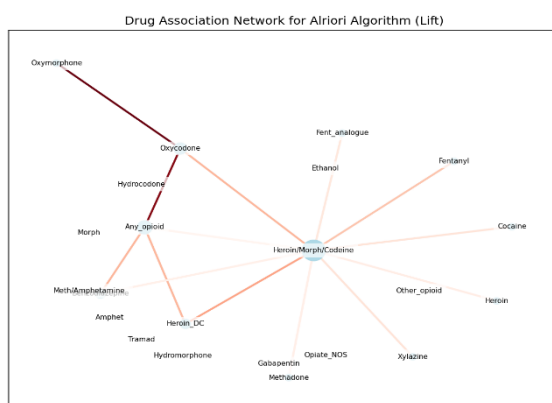


Figure 68: Network graph for Apriori Algorithm

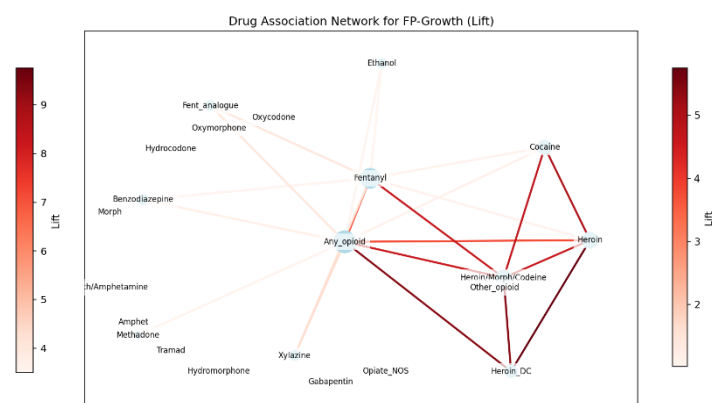


Figure 69: Network graph for FP-growth Algorithm

Moreover, according to the analysis of the multi-dimensional association rule using Apriori and FP-growth, they have produced a slightly different result with the Apriori algorithm consisting of more drug types, but the most frequently used substance that appears in both algorithms is the same, which is “Fentanyl” and “Any\_opiod”.

In terms of performance analysis of the Apriori and FP-growth algorithms, the time complexity and space complexity of FP-growth is significantly smaller than the Apriori Algorithm, which means that FP-growth typically requires less time to execute the task with minimal memory usage compared to the Apriori Algorithms. This might have been because of the divide and conquer method used by FP-growth to eliminate the need for candidate generation can remove the repeated scanning process for finding frequent items that would cause the increase in execution time.

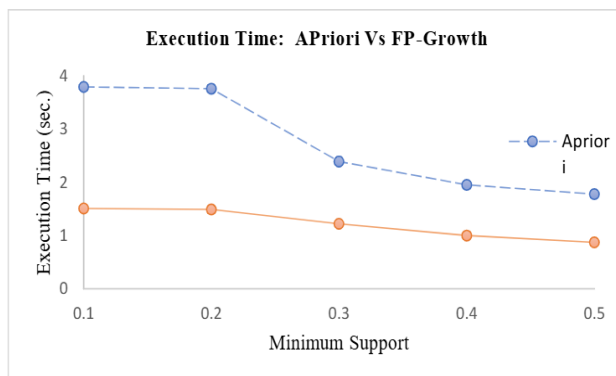


Figure 70: Execution time of Apriori and FP-growth

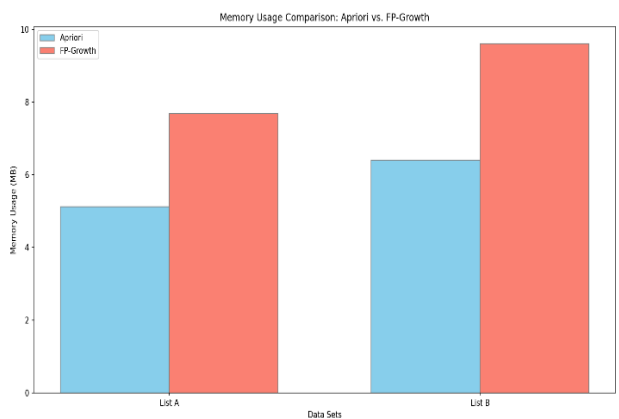


Figure 71: Memory Usage of Apriori and FP-growth

**Table 3: Comparison of Apriori Algorithm and FP-growth Algorithm**

| Criteria         | Apriori Algorithm | FP-Growth Algorithm |
|------------------|-------------------|---------------------|
| Rules produce    | Lesser            | More                |
| Association Rule | Less Strong       | Stronger            |
| Execution time   | Slower            | Faster              |
| Memory usage     | higher            | lower               |

### Strengths and Weaknesses of both Algorithm

#### Apriori Algorithm

| Strength                                            | Weakness                                                                                                                         |
|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Less complexity compared to FP-growth               | Require a considerable amount of execution time to handle a large dataset due to a lot of candidates being generated.            |
| Relatively simple to be interpreted and implemented | Require more memory space as the number of substances increases.                                                                 |
| Can efficiently manage the categorical data         | Highly sensitive to the minimum support threshold as a slight change in the value will result in different rules being produced. |
| Can efficiently handle the small dataset            | The number of itemset might grow exponentially as dealing with multi-dimensional association rule.                               |

#### FP-growth Algorithm

| Strength                                                                                                                        | Weakness                                                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Can effectively handle a large data set without increasing the time complexity and space complexity                             | The FP-growth might not be able to handle the categorical data well, therefore, needs to perform a lot of preprocessing steps.                                    |
| Can generate the frequent itemset quickly                                                                                       | Too many items are being generated, which might render the process of analysis challenging and hence, lead to difficulty in making useful and meaningful insight. |
| The use of data structure, FP-tree has reduced the need for multiple scans on candidate items, hence, reducing the memory usage | The CP-growth is relatively complex compared to the Apriori Algorithm.                                                                                            |
| The FP-growth is scalable to handle millions of substances.                                                                     | Cannot efficiently handle the real-time data                                                                                                                      |

### 3.2 Recommendation and Conclusion

- The FP-Growth algorithm is more suitable for handling the selected dataset since it involves a large volume of data that might not be efficient if handled by the Apriori Algorithm.
- A risk assessment should be developed especially for adults who are between the ages range 30 and 50.
- Extra care should be paid by the residence cities that are identified as high-risk such as Hartford.
- A prevention measure should be established on the combination of drugs that are identified as frequent itemset with high confidence and lift.

In conclusion, the implementation of both the Apriori Algorithm and the FP-growth Algorithm in the Accidental Drug-Related Death datasets has assisted in revealing various of variable insights by identifying the frequent items and their association. This revealed pattern be of great assistance in conducting risk assessment and, therefore, making an informed medical decision to overcome the drug fatality issue. The performance of both algorithms has also been evaluated to determine which algorithm performs better in certain circumstances. Which, in this case, the FP-growth algorithm is well suited to process the selected dataset that consists of ten thousand of records that might not be efficient if the Apriori Algorithm is utilised.

## References

- Alam, B. (2022, February 20). *Implementation of FP-growth algorithm using Python*. Retrieved from [https://hands-on.cloudy: https://hands-on.cloud/implementation-of-fp-growth-algorithm-using-python/](https://hands-on.cloudy:https://hands-on.cloud/implementation-of-fp-growth-algorithm-using-python/)
- AspiringYouths. (n.d.). *Advantages and Disadvantages of Apriori Algorithm*. Retrieved from [https://aspiringyouths.com: https://aspiringyouths.com/advantages-disadvantages/apriori-algorithm/#google\\_vignette](https://aspiringyouths.com:https://aspiringyouths.com/advantages-disadvantages/apriori-algorithm/#google_vignette)
- Çoğalan, A. (2023, Mar 22). *FP-Growth Algorithm: How to Analyze User Behavior and Outrank Your Competitors*. Retrieved from <https://medium.com/@anilcogalan/fp-growth-algorithm-how-to-analyze-user-behavior-and-outrank-your-competitors-c39af08879db>
- geeksforgeeks. (Frequent Pattern Growth Algorithm, May 02). Retrieved from [https://www.geeksforgeeks.org: https://www.geeksforgeeks.org/frequent-pattern-growth-algorithm/](https://www.geeksforgeeks.org:https://www.geeksforgeeks.org/frequent-pattern-growth-algorithm/)
- Imameev, D. (2021). *FP-Growth. Association Rules*. Retrieved from [https://www.kaggle.com: https://www.kaggle.com/code/dinirimameev/fp-growth-association-rules/notebook](https://www.kaggle.com:https://www.kaggle.com/code/dinirimameev/fp-growth-association-rules/notebook)
- Kaggle. (2020, 5 2). *Market Basket Analysis*. Retrieved from [https://www.kaggle.com: https://www.kaggle.com/code/xvivancos/market-basket-analysis/report](https://www.kaggle.com:https://www.kaggle.com/code/xvivancos/market-basket-analysis/report)
- Roshan, B. (2020, Dec 23). *Market Basket Analysis - Association Rule Mining with visualizations*. Retrieved from [https://medium.com: https://medium.com/analytics-vidhya/market-basket-analysis-association-rule-mining-with-visualizations-cda24d537019](https://medium.com:https://medium.com/analytics-vidhya/market-basket-analysis-association-rule-mining-with-visualizations-cda24d537019)
- Elavarasan, D., Vincent, D. R., Sharma, V., Zomaya, A. Y., & Srinivasan, K. (2018). Forecasting yield by integrating agrarian factors and machine learning models: A survey. *Computers and Electronics in Agriculture*, 155, 257-282. <https://doi.org/10.1016/j.compag.2018.10.024>
- Kotsiantis, S., & Kanellopoulos, D. (2005). Association rules mining: A recent overview. *GESTS International Transactions on Computer Science and Engineering*, 32, 71–82
- Kavitha, B., Konda, S., Reddy, B., & Govardhan, D. (2011). Mining negative association rules. *International Journal of Engineering and Technology*, 3, 100–105.
- Mahmood, S., Shahbaz, M., & Guergachi, A. (2014). Negative and positive association rules mining

from text using frequent and infrequent itemsets. *TheScientificWorldJournal*, 2014, 973750.  
<https://doi.org/10.1155/2014/973750>

Hahsler, M., & Chelluboina, S. (2015). *Visualizing association rules: Introduction to the R-extension package arulesViz*.

Lawal, M., Ogedengbe, M., & Ng. (2024). *FP-Growth Algorithm: Mining Association Rules without Candidate Sets Generation*. *Kasu Journal of Computer Science*, 1(2), 392–411.  
<https://doi.org/10.47514/kjcs/2024.1.2.0016>

## Appendix

## Appendix A

| Sample of dataset: |            |               |      |        |       |           |           |            |           |             |               |              |
|--------------------|------------|---------------|------|--------|-------|-----------|-----------|------------|-----------|-------------|---------------|--------------|
|                    | Date       | Date_type     | Age  | Sex    | Race  | Ethnicity | Res_city  | Res_county | Res_state | Injury_city | Injury_county | Injury_state |
| 0                  | 05-29-2012 | Date_of_death | 37.0 | Male   | Black | Hispanic  | STAMFORD  | FAIRFIELD  | CT        | STAMFORD    | NEW_HAVEN     | CT           |
| 1                  | 06-27-2012 | Date_of_death | 37.0 | Male   | White | Hispanic  | NORWICH   | NEW_LONDON | CT        | NORWICH     | NEW_HAVEN     | CT           |
| 2                  | 03-24-2014 | Date_of_death | 28.0 | Male   | White | Hispanic  | HEBRON    | NEW_HAVEN  | CT        | HEBRON      | NEW_HAVEN     | CT           |
| 3                  | 12-31-2014 | Date_of_death | 26.0 | Female | White | Hispanic  | BALTIC    | NEW_HAVEN  | CT        | HARTFORD    | NEW_HAVEN     | CT           |
| 4                  | 01-16-2016 | Date_of_death | 41.0 | Male   | White | Hispanic  | SHELTON   | FAIRFIELD  | CT        | SHELTON     | NEW_HAVEN     | CT           |
| 5                  | 06-13-2017 | Date_reported | 57.0 | Male   | White | Hispanic  | BLANDFORD | HAMPDEN    | MA        | ENFIELD     | HARTFORD      | CT           |
| 6                  | 10-20-2015 | Date_reported | 26.0 | Male   | White | Hispanic  | DANBURY   | FAIRFIELD  | CT        | DANBURY     | NEW_HAVEN     | CT           |
| 7                  | 2-2-2017   | Date_reported | 64.0 | Male   | White | Hispanic  | MILFORD   | NEW_HAVEN  | CT        | MILFORD     | NEW_HAVEN     | CT           |
| 8                  | 7-3-2018   | Date_of_death | 33.0 | Male   | White | Hispanic  | HARTFORD  | NEW_HAVEN  | CT        | HARTFORD    | HARTFORD      | CT           |
| 9                  | 5-8-2013   | Date_of_death | 23.0 | Male   | White | Hispanic  | BETHEL    | FAIRFIELD  | CT        | BETHEL      | NEW_HAVEN     | CT           |

<class 'pandas.core.frame.DataFrame'>

| Injury_place | Injury_description | Death_city  | Death_county | Death_state | Location  | Other_location     | Cause_of_death                      |
|--------------|--------------------|-------------|--------------|-------------|-----------|--------------------|-------------------------------------|
| Residence    | Used_Cocaine       | HARTFORD    | HARTFORD     | CT          | Residence | unknown            | Cocaine_Toxicity                    |
| Residence    | Drug_Use           | NORWICH     | NEW_LONDON   | CT          | Hospital  | unknown            | Heroin_Toxicity                     |
| Residence    | Drug_Use           | MARLBOROUGH | HARTFORD     | CT          | Hospital  | unknown            | Heroin_Intoxication                 |
| Residence    | Substance_Abuse    | BALTIC      | NEW_LONDON   | CT          | Residence | unknown            | Acute_Heroin_Intoxication           |
| Residence    | Drug_Use           | BRIDGEPORT  | HARTFORD     | CT          | Hospital  | unknown            | Acute_Fentanyl_Intoxication         |
| In_Vehicle   | Drug_Abuse         | ENFIELD     | HARTFORD     | CT          | Other     | Roadway_in_vehicle | Acute_Intoxication_Cocaine_Toxicity |
| Residence    | Substance_Abuse    | DANBURY     | FAIRFIELD    | CT          | Hospital  | unknown            | Acute_Opioid_Intoxication           |
| Residence    | Ingestion          | MILFORD     | NEW_HAVEN    | CT          | Residence | unknown            | Acute_Oxycodone_Intoxication        |
| Building     | Drug_Use           | HARTFORD    | HARTFORD     | CT          | Other     | unknown            | Acute_Cocaine_Intoxication          |
| Residence    | Inhalation         | BETHEL      | FAIRFIELD    | CT          | Residence | unknown            | Heroin_Intoxication                 |

[illegible]

| ResidenceCityGeo                     | InjuryCityGeo                        | DeathCityGeo                            |
|--------------------------------------|--------------------------------------|-----------------------------------------|
| STAMFORD, CT (41.051924, -73.539475) | STAMFORD, CT (41.051924, -73.539475) | CT (41.575155, -72.738288)              |
| NORWICH, CT (41.524304, -72.075821)  | NORWICH, CT (41.524304, -72.075821)  | Norwich, CT (41.524304, -72.075821)     |
| HEBRON, CT (41.658069, -72.366324)   | HEBRON, CT (41.658069, -72.366324)   | Marlborough, CT (41.632043, -72.461309) |
| BALTIC, CT (41.617221, -72.085031)   | CT (41.575155, -72.738288)           | Baltic, CT (41.617221, -72.085031)      |
| SHELTON, CT (41.316843, -73.092968)  | SHELTON, CT (41.316843, -73.092968)  | Bridgeport, CT (41.179195, -73.189476)  |
| HARTFORD, CT (41.765775, -72.673356) | ENFIELD, CT (41.976501, -72.591985)  | Enfield, CT (41.976501, -72.591985)     |
| DANBURY, CT (41.393666, -73.451539)  | DANBURY, CT (41.393666, -73.451539)  | Danbury, CT (41.393666, -73.451539)     |
| MILFORD, CT (41.224276, -73.057564)  | MILFORD, CT (41.224276, -73.057564)  | Milford, CT (41.224276, -73.057564)     |
| CT (41.575155, -72.738288)           | HARTFORD, CT (41.765775, -72.673356) | Hartford, CT (41.765775, -72.673356)    |
| BETHEL, CT (41.371727, -73.407448)   | BETHEL, CT (41.371727, -73.407448)   | Bethel, CT (41.371727, -73.407448)      |

[illegible]

RangeIndex: 11981 entries, 0 to 11980

Data columns (total 48 columns):

| #  | Column             | Non-Null Count | Dtype   |
|----|--------------------|----------------|---------|
| 0  | Date               | 11981 non-null | object  |
| 1  | Date_type          | 11981 non-null | object  |
| 2  | Age                | 11981 non-null | float64 |
| 3  | Sex                | 11981 non-null | object  |
| 4  | Race               | 11981 non-null | object  |
| 5  | Ethnicity          | 11981 non-null | object  |
| 6  | Res_city           | 11981 non-null | object  |
| 7  | Res_county         | 11981 non-null | object  |
| 8  | Res_state          | 11981 non-null | object  |
| 9  | Injury_city        | 11981 non-null | object  |
| 10 | Injury_county      | 11981 non-null | object  |
| 11 | Injury_state       | 11981 non-null | object  |
| 12 | Injury_place       | 11981 non-null | object  |
| 13 | Injury_description | 11981 non-null | object  |
| 14 | Death_city         | 11981 non-null | object  |
| 15 | Death_county       | 11981 non-null | object  |
| 16 | Death_state        | 11981 non-null | object  |
| 17 | Location           | 11981 non-null | object  |
| 18 | Other_location     | 11981 non-null | object  |
| 19 | Cause_of_death     | 11981 non-null | object  |
| 20 | Death_manner       | 11981 non-null | object  |
| 21 | Other_condition    | 11981 non-null | object  |

|    |                      |                |        |
|----|----------------------|----------------|--------|
| 22 | Heroin               | 11981 non-null | object |
| 23 | Heroin_DC            | 11981 non-null | object |
| 24 | Cocaine              | 11981 non-null | object |
| 25 | Fentanyl             | 11981 non-null | object |
| 26 | Fent_analogue        | 11981 non-null | object |
| 27 | Oxycodone            | 11981 non-null | object |
| 28 | Oxymorphone          | 11981 non-null | object |
| 29 | Ethanol              | 11981 non-null | object |
| 30 | Hydrocodone          | 11981 non-null | object |
| 31 | Benzodiazepine       | 11981 non-null | object |
| 32 | Methadone            | 11981 non-null | object |
| 33 | Meth/Amphetamine     | 11981 non-null | object |
| 34 | Amphet               | 11981 non-null | object |
| 35 | Tramad               | 11981 non-null | object |
| 36 | Hydromorphone        | 11981 non-null | object |
| 37 | Morph                | 11981 non-null | object |
| 38 | Xylazine             | 11981 non-null | object |
| 39 | Gabapentin           | 11981 non-null | object |
| 40 | Opiate_NOS           | 11981 non-null | object |
| 41 | Heroin/Morph/Codeine | 11981 non-null | object |
| 42 | Other_opioid         | 11981 non-null | object |
| 43 | Any_opioid           | 11981 non-null | object |
| 44 | Other                | 11981 non-null | object |
| 45 | ResidenceCityGeo     | 11981 non-null | object |
| 46 | InjuryCityGeo        | 11981 non-null | object |
| 47 | DeathCityGeo         | 11981 non-null | object |

## Appendix B

| Table after removing the entries that contain one or zero: |        |           |         |          |         |               |           |             |             |                |       |
|------------------------------------------------------------|--------|-----------|---------|----------|---------|---------------|-----------|-------------|-------------|----------------|-------|
|                                                            | Heroin | Heroin_DC | Cocaine | Fentanyl | Ethanol | Fent_analogue | Oxycodone | Oxymorphone | Hydrocodone | Benzodiazepine | Morph |
| 0                                                          | False  | False     | False   | True     | False   | False         | False     | False       | False       | False          | False |
| 1                                                          | True   | True      | False   | False    | False   | False         | False     | False       | False       | False          | False |
| 2                                                          | True   | True      | False   | False    | False   | False         | False     | False       | False       | False          | False |
| 3                                                          | True   | True      | True    | False    | False   | False         | False     | False       | False       | False          | False |
| 4                                                          | True   | False     | False   | False    | True    | False         | False     | False       | False       | False          | False |
| 5                                                          | False  | False     | True    | True     | False   | False         | False     | False       | False       | False          | False |
| 6                                                          | False  | False     | True    | True     | False   | False         | False     | False       | False       | False          | False |
| 7                                                          | False  | False     | False   | True     | False   | False         | False     | False       | False       | False          | False |
| 8                                                          | False  | False     | False   | True     | False   | False         | False     | False       | False       | False          | False |
| 9                                                          | False  | False     | False   | False    | False   | False         | True      | False       | False       | False          | False |

[illegible]

## Appendix C

```

def negative_rules(ar_results, sub, min_support, min_confidence):
    # create a list for storing negative rule
    neg_rules = []

    # loop through the item inside the ar_result and store it inside the pair list
    for item in ar_results:
        pair = item.items
        items = list(pair)

        # Skip the items that have the item less than 2
        if len(items) < 2:
            continue

        # store the first index inside the items to A and the second position of items will be store in b
        A = {items[0]}
        B = {items[1]}

        # Call of function to compute the negative support
        (supp_A, supp_B, supp_AB, supp_nA, support_not_B, supp_A_nB,
         supp_nA_B) = compute_neg_support(A, B, sub)

        # Formula for calculating the negative confidence
        conf_A_nB = supp_A_nB / supp_A if supp_A else 0
        conf_nA_B = supp_nA_B / supp_nA if supp_nA else 0

        conv_A_nB = (1 - supp_B) / max(1 - conf_A_nB, 1e-10)
        conv_nA_B = (1 - supp_A) / max(1 - conf_nA_B, 1e-10)

        # if the confidence(A->~B) >= 0.7 and support(A -> ~B) >=0.005
        # the rule will be appended to neg_rules
        if conf_A_nB >= min_confidence and supp_A_nB >= min_support:
            neg_rules.append(
                {'Rule': f"{A} → NOT_{B}",
                 'Support': supp_A_nB,
                 'Confidence': conf_A_nB,
                 'Conviction': conv_A_nB
                })

        # if the confidence(~A -> B) >= 0.7 and support(~A->B) >=0.05
        # append it to negative rules
        if conf_nA_B >= min_confidence and supp_nA_B >= min_support:
            (neg_rules.append
             ({'Rule': f"NOT_{A} → {B}",
              'Support': supp_nA_B,
              'Confidence': conf_nA_B,
              'Conviction': conv_nA_B
              })
            )

    # return the neg_rules in a data frame format
    return pd.DataFrame(neg_rules)

```

## Appendix D

| Overall Results with FP-Growth: |                                               |                                            |          |            |          |
|---------------------------------|-----------------------------------------------|--------------------------------------------|----------|------------|----------|
|                                 | antecedents                                   | consequents                                | support  | confidence | lift     |
| 0                               | (Any_opioid, Heroin_DC)                       | (Heroin/Morph/Codeine)                     | 0.061598 | 1.000000   | 5.440963 |
| 1                               | (Any_opioid, Heroin_DC, Heroin)               | (Heroin/Morph/Codeine)                     | 0.061598 | 1.000000   | 5.440963 |
| 2                               | (Any_opioid, Heroin_DC)                       | (Heroin, Heroin/Morph/Codeine)             | 0.061598 | 1.000000   | 5.751800 |
| 3                               | (Xylazine, Fentanyl)                          | (Any_opioid)                               | 0.089642 | 1.000000   | 1.357159 |
| 4                               | (Any_opioid, Heroin_DC, Heroin/Morph/Codeine) | (Heroin)                                   | 0.061598 | 1.000000   | 3.348519 |
| 5                               | (Heroin_DC)                                   | (Heroin/Morph/Codeine)                     | 0.061764 | 1.000000   | 5.440963 |
| 6                               | (Heroin_DC, Heroin/Morph/Codeine)             | (Heroin)                                   | 0.061764 | 1.000000   | 3.348519 |
| 7                               | (Heroin_DC)                                   | (Heroin)                                   | 0.061764 | 1.000000   | 3.348519 |
| 8                               | (Heroin_DC, Heroin)                           | (Heroin/Morph/Codeine)                     | 0.061764 | 1.000000   | 5.440963 |
| 9                               | (Heroin_DC)                                   | (Heroin, Heroin/Morph/Codeine)             | 0.061764 | 1.000000   | 5.751800 |
| 10                              | (Any_opioid, Heroin_DC)                       | (Heroin)                                   | 0.061598 | 1.000000   | 3.348519 |
| 11                              | (Fentanyl, Heroin, Heroin/Morph/Codeine)      | (Any_opioid)                               | 0.116351 | 0.999283   | 1.356186 |
| 12                              | (Xylazine)                                    | (Any_opioid)                               | 0.089892 | 0.999072   | 1.355900 |
| 13                              | (Heroin, Heroin/Morph/Codeine)                | (Any_opioid)                               | 0.173608 | 0.998560   | 1.355204 |
| 14                              | (Cocaine, Heroin/Morph/Codeine)               | (Any_opioid)                               | 0.057842 | 0.998559   | 1.355203 |
| 15                              | (Cocaine, Heroin, Heroin/Morph/Codeine)       | (Any_opioid)                               | 0.056673 | 0.998529   | 1.355163 |
| 16                              | (Heroin_DC)                                   | (Any_opioid, Heroin)                       | 0.061598 | 0.997297   | 5.099709 |
| 17                              | (Heroin_DC, Heroin)                           | (Any_opioid)                               | 0.061598 | 0.997297   | 1.353491 |
| 18                              | (Heroin_DC, Heroin, Heroin/Morph/Codeine)     | (Any_opioid, Heroin)                       | 0.061598 | 0.997297   | 1.353491 |
| 19                              | (Heroin_DC)                                   | (Any_opioid)                               | 0.061598 | 0.997297   | 1.353491 |
| 20                              | (Heroin_DC, Heroin/Morph/Codeine)             | (Any_opioid)                               | 0.061598 | 0.997297   | 1.353491 |
| 21                              | (Heroin_DC, Heroin)                           | (Any_opioid, Heroin/Morph/Codeine)         | 0.061598 | 0.997297   | 5.443562 |
| 22                              | (Heroin_DC, Heroin/Morph/Codeine)             | (Any_opioid, Heroin)                       | 0.061598 | 0.997297   | 5.099709 |
| 23                              | (Heroin_DC)                                   | (Any_opioid, Heroin, Heroin/Morph/Codeine) | 0.061598 | 0.997297   | 5.744528 |
| 24                              | (Heroin_DC)                                   | (Any_opioid, Heroin/Morph/Codeine)         | 0.061598 | 0.997297   | 5.443562 |
| 25                              | (Any_opioid, Xylazine)                        | (Fentanyl)                                 | 0.089642 | 0.997214   | 1.484362 |
| 26                              | (Fentanyl, Heroin/Morph/Codeine)              | (Any_opioid)                               | 0.118104 | 0.997181   | 1.353333 |
| 27                              | (Heroin/Morph/Codeine)                        | (Any_opioid)                               | 0.183207 | 0.996821   | 1.352845 |
| 28                              | (Xylazine)                                    | (Any_opioid, Fentanyl)                     | 0.089642 | 0.996289   | 1.648694 |
| 29                              | (Xylazine)                                    | (Fentanyl)                                 | 0.089642 | 0.996289   | 1.482985 |
| 30                              | (Fent_analogue)                               | (Fentanyl)                                 | 0.080294 | 0.987680   | 1.470169 |
| 31                              | (Any_opioid, Fent_analogue)                   | (Fentanyl)                                 | 0.068776 | 0.986826   | 1.468899 |
| 32                              | (Any_opioid, Fentanyl, Heroin/Morph/Codeine)  | (Heroin)                                   | 0.116351 | 0.985159   | 3.298823 |
| 33                              | (Fentanyl, Heroin/Morph/Codeine)              | (Heroin)                                   | 0.116434 | 0.983087   | 3.291884 |
| 34                              | (Fentanyl, Heroin/Morph/Codeine)              | (Any_opioid, Heroin)                       | 0.116351 | 0.982382   | 5.023439 |
| 35                              | (Cocaine, Heroin/Morph/Codeine)               | (Heroin)                                   | 0.056757 | 0.979827   | 3.280969 |
| 36                              | (Cocaine, Any_opioid, Heroin/Morph/Codeine)   | (Heroin)                                   | 0.056673 | 0.979798   | 3.280872 |
| 37                              | (Cocaine, Heroin/Morph/Codeine)               | (Any_opioid, Heroin)                       | 0.056673 | 0.978386   | 5.003007 |
| 38                              | (Any_opioid, Heroin/Morph/Codeine)            | (Heroin)                                   | 0.173608 | 0.947608   | 3.173084 |
| 39                              | (Heroin/Morph/Codeine)                        | (Heroin)                                   | 0.173859 | 0.945958   | 3.167559 |
| 40                              | (Heroin/Morph/Codeine)                        | (Any_opioid, Heroin)                       | 0.173608 | 0.944596   | 4.830219 |
| 41                              | (Cocaine, Fentanyl, Ethanol)                  | (Any_opioid)                               | 0.067524 | 0.925629   | 1.256226 |
| 42                              | (Cocaine, Fentanyl)                           | (Any_opioid)                               | 0.256156 | 0.923841   | 1.253799 |
| 43                              | (Fentanyl, Ethanol)                           | (Any_opioid)                               | 0.164844 | 0.908046   | 1.232363 |
| 44                              | (Fentanyl)                                    | (Any_opioid)                               | 0.604290 | 0.899491   | 1.220752 |
| 45                              | (Cocaine, Any_opioid, Ethanol)                | (Fentanyl)                                 | 0.067524 | 0.897891   | 1.336518 |
| 46                              | (Cocaine, Any_opioid)                         | (Fentanyl)                                 | 0.256156 | 0.897368   | 1.335740 |
| 47                              | (Any_opioid, Heroin)                          | (Heroin/Morph/Codeine)                     | 0.173608 | 0.887751   | 4.830219 |
| 48                              | (Cocaine, Any_opioid, Heroin)                 | (Heroin/Morph/Codeine)                     | 0.056673 | 0.870513   | 4.736428 |
| 49                              | (Benzodiazepine, Fentanyl)                    | (Any_opioid)                               | 0.115767 | 0.866334   | 1.175752 |
| 50                              | (Fent_analogue)                               | (Any_opioid)                               | 0.069694 | 0.857290   | 1.163478 |
| 51                              | (Fentanyl, Fent_analogue)                     | (Any_opioid)                               | 0.068776 | 0.856549   | 1.162473 |
| 52                              | (Any_opioid, Heroin, Fentanyl)                | (Heroin/Morph/Codeine)                     | 0.116351 | 0.851038   | 4.630465 |
| 53                              | (Fent_analogue)                               | (Any_opioid, Fentanyl)                     | 0.068776 | 0.845996   | 1.399983 |
| 54                              | (Any_opioid, Ethanol)                         | (Fentanyl)                                 | 0.164844 | 0.821206   | 1.222371 |
| 55                              | (Any_opioid)                                  | (Fentanyl)                                 | 0.604290 | 0.820118   | 1.220752 |
| 56                              | (Heroin, Fentanyl)                            | (Any_opioid)                               | 0.136716 | 0.810891   | 1.100508 |
| 57                              | (Cocaine, Ethanol)                            | (Any_opioid)                               | 0.075202 | 0.788967   | 1.070753 |
| 58                              | (Methadone)                                   | (Any_opioid)                               | 0.070695 | 0.785714   | 1.066339 |
| 59                              | (Cocaine, Ethanol)                            | (Fentanyl)                                 | 0.072949 | 0.765324   | 1.139191 |
| 60                              | (Ethanol)                                     | (Any_opioid)                               | 0.200734 | 0.751328   | 1.019671 |
| 61                              | (Cocaine)                                     | (Any_opioid)                               | 0.285452 | 0.747051   | 1.013867 |
| 62                              | (Cocaine)                                     | (Fentanyl)                                 | 0.277272 | 0.725644   | 1.080127 |
| 63                              | (Cocaine, Ethanol)                            | (Any_opioid, Fentanyl)                     | 0.067524 | 0.708406   | 1.172295 |
| 64                              | (Any_opioid, Benzodiazepine)                  | (Fentanyl)                                 | 0.115767 | 0.707292   | 1.052810 |
| 65                              | (Any_opioid, Heroin)                          | (Fentanyl)                                 | 0.136716 | 0.699104   | 1.040621 |
| 66                              | (Fentanyl, Heroin)                            | (Heroin/Morph/Codeine)                     | 0.116434 | 0.690594   | 3.757497 |
| 67                              | (Fentanyl, Heroin)                            | (Any_opioid, Heroin/Morph/Codeine)         | 0.116351 | 0.690099   | 3.766777 |
| 68                              | (Ethanol)                                     | (Fentanyl)                                 | 0.181537 | 0.679475   | 1.011404 |
| 69                              | (Cocaine)                                     | (Any_opioid, Fentanyl)                     | 0.256156 | 0.670380   | 1.109368 |
| 70                              | (Heroin, Heroin/Morph/Codeine)                | (Any_opioid, Fentanyl)                     | 0.116351 | 0.669227   | 1.107460 |
| 71                              | (Heroin/Morph/Codeine)                        | (Any_opioid, Fentanyl)                     | 0.118104 | 0.642598   | 1.063393 |
| 72                              | (Any_opioid, Heroin/Morph/Codeine)            | (Fentanyl, Heroin)                         | 0.116351 | 0.635080   | 3.766777 |
| 73                              | (Heroin/Morph/Codeine)                        | (Fentanyl, Heroin)                         | 0.116434 | 0.633515   | 3.757497 |
| 74                              | (Heroin/Morph/Codeine)                        | (Any_opioid, Heroin, Fentanyl)             | 0.116351 | 0.633061   | 4.630465 |
| 75                              | (Ethanol)                                     | (Any_opioid, Fentanyl)                     | 0.164844 | 0.616995   | 1.021024 |